🔗 Jenkins Integrations: Complete Interview Mastery Guide

Category 6 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic: Simple Definition → Why it exists → Internals → Key concepts → Short interview answer → Deep dive → Real-world example → Interview Q&A → Gotchas → Connections.
>
> ⚠ = Frequently misunderstood or heavily tested.
>
> Category 6 is where pipeline knowledge meets the real world. Interviewers ask about integrations because they reveal whether you've actually connected Jenkins to the ecosystem of tools a modern engineering org uses. The ⭐ topics — Docker, Kubernetes, Vault, Terraform — are where senior/staff-level interviews spend most time. They test design thinking: not "show me the syntax" but "explain why you designed it that way and what the failure modes are."

📑 TABLE OF CONTENTS

1. Topic 6.1 — Jenkins + Git
2. Topic 6.2 — Jenkins + Docker ⚠
3. Topic 6.3 — Jenkins + Kubernetes
4. Topic 6.4 — Jenkins + SonarQube
5. Topic 6.5 — Jenkins + Nexus/Artifactory
6. Topic 6.6 — Jenkins + Vault/Secrets Managers
7. Topic 6.7 — Jenkins + AWS/GCP/Azure
8. Topic 6.8 — Jenkins + Terraform
9. Category 6 Summary & Self-Quiz

Topic 6.1 — Jenkins + Git

🟢 Beginner | The Foundation of Every Pipeline

📌 What It Is — In Simple Terms

Every Jenkins pipeline starts with source code from a Git repository. The Jenkins+Git integration controls: how Jenkins authenticates to the repository, how it's notified when code changes (webhooks vs polling), how it clones (shallow vs deep, sparse vs full), and how branching strategy (GitFlow vs trunk-based) affects pipeline design.

⚙️ Authentication Methods

```
Method 1: HTTPS with Username/Password (or PAT)
  Store as: Jenkins Username+Password credential
  credentialsId: 'github-https-creds'
  URL format: https://github.com/myorg/myapp.git

  Pros:  Simple, works everywhere
  Cons:  PAT leakage if logs capture URL (use credential binding, not
inline URL)

Method 2: SSH Key Pair
  Store as: Jenkins SSH private key credential
  credentialsId: 'github-ssh-key'
  URL format: git@github.com:myorg/myapp.git

  Pros:  No password expiry, more secure, standard for server-to-server
auth
  Cons:  Agent needs correct SSH host key fingerprint for github.com

Method 3: GitHub App (for Organization-level access)
  Store as: Jenkins GitHub App credential (App ID + private key .pem)
  Provides: short-lived, scoped tokens auto-rotated by GitHub
  Best for: Organization Folders, high-volume API access
  Rate limit: 5000 req/hr per installation (vs 5000/hr per PAT)
```

```
// ── CHECKOUT REFERENCE ──────────────────────────────────────────────
// Simple (Multi-Branch Pipeline — uses repo already configured at job
level):
checkout scm

// Full control:
checkout([
    $class: 'GitSCM',
    branches: [[name: env.BRANCH_NAME ?: '*/main']],
    userRemoteConfigs: [[
        url:           'https://github.com/myorg/myapp.git',
        credentialsId: 'github-creds'
    ]],
    extensions: [
        // Shallow clone — only fetch last N commits (fast for big repos)
        [$class: 'CloneOption', shallow: true, depth: 10, noTags: false,
timeout: 15],

        // Clean workspace before checkout (git clean -fdx)
        [$class: 'CleanBeforeCheckout'],

        // Sparse checkout — monorepo: only fetch what this service needs
        [$class: 'SparseCheckoutPaths', sparseCheckoutPaths: [
            [path: 'services/auth/'],
```

```
            [path: 'shared/'],
            [path: 'deploy/']
        ]],

        // Set the committer identity for any git operations in the
pipeline
        [$class: 'UserIdentity', name: 'Jenkins CI', email:
'jenkins@example.com'],

        // Timeout if checkout takes longer than N minutes
        [$class: 'CheckoutOption', timeout: 10],

        // Submodule support:
        [$class: 'SubmoduleOption',
         recursiveSubmodules: true,
         disableSubmodules: false,
         trackingSubmodules: false,
         reference: '',
         timeout: 10,
         shallow: true]
    ]
])
```

⚙️ Webhooks vs Polling — The Critical Choice

```
POLLING (SCM Poll):
  Jenkins Controller → (every N minutes) → Git API → "any changes?"

  Configuration:
    triggers { pollSCM('H/5 * * * *') }   // Poll every 5 minutes

  Problems:
    ❌ Controller load: each poll = outbound HTTP request
    ❌ Latency: up to 5 minutes before build starts
    ❌ Scales poorly: 500 repos × every 5 min = 100 polls/minute on
Controller
    ❌ Missed changes: if Jenkins is down during a push, it polls next
cycle
    ❌ Rate limits: GitHub API rate limits affect polling (5000/hr)

WEBHOOKS (Push-based):
  Git server → (on push/PR event) → Jenkins webhook endpoint → trigger
build

  Latency: Build starts within seconds of push
  Controller load: Only fires when events actually happen
  No rate limiting concern

  GitHub webhook URL:
    https://jenkins.example.com/github-webhook/        (GitHub plugin)
```

```
    https://jenkins.example.com/generic-webhook-trigger/invoke?token=TOKEN

  GitLab webhook URL:
    https://jenkins.example.com/project/JOBNAME        (GitLab plugin)

  ✅ Always prefer webhooks over polling

HYBRID (best practice):
  Primary:  webhook for immediate builds
  Fallback: pollSCM with long interval (1 hour) for when webhook delivery
fails

  triggers {
      githubPush()                    // Primary - immediate
      pollSCM('H * * * *')            // Fallback - hourly (H spreads
load)
  }
```

⚙️ Git Environment Variables Available in Pipeline

```
// These are set automatically by Git checkout:
stage('Use Git Variables') {
    steps {
        echo "Full commit SHA:  ${env.GIT_COMMIT}"         //
abc1234def5678...
        echo "Short SHA:        ${env.GIT_COMMIT.take(7)}" // abc1234
        echo "Git branch:       ${env.GIT_BRANCH}"         // origin/main
        echo "Clean branch:     ${env.BRANCH_NAME}"        // main (Multi-
Branch only)
        echo "Previous commit:  ${env.GIT_PREVIOUS_COMMIT}"
        echo "Successful commit:${env.GIT_PREVIOUS_SUCCESSFUL_COMMIT}"
        echo "Remote URL:       ${env.GIT_URL}"
        echo "Author name:      ${env.GIT_AUTHOR_NAME}"
        echo "Author email:     ${env.GIT_AUTHOR_EMAIL}"
        echo "Committer:        ${env.GIT_COMMITTER_NAME}"

        // Get more git info via shell:
        script {
            env.COMMIT_MSG    = sh(script: 'git log -1 --format=%s',
returnStdout: true).trim()
            env.COMMIT_AUTHOR = sh(script: 'git log -1 --format=%an',
returnStdout: true).trim()
            env.CHANGED_FILES = sh(script: 'git diff --name-only HEAD~1
HEAD', returnStdout: true).trim()
            env.TAG_NAME      = sh(script: 'git describe --tags --exact-
match 2>/dev/null || echo ""', returnStdout: true).trim()
        }
    }
}
```

⚙️ Branch Strategy Impact on Pipeline Design

```
TRUNK-BASED DEVELOPMENT:
  Branches: main only (+ short-lived feature branches, < 2 days)
  Pipeline design:
    Every commit to main → full CI + auto-deploy to staging
    Feature branches → lightweight CI (build + unit test)
    Tags (vX.Y.Z) → production deploy

  Jenkins setup: Multi-Branch Pipeline
  when conditions:
    Deploy staging: when { branch 'main' }
    Deploy prod:    when { tag 'v*' }

GITFLOW:
  Branches: main, develop, release/X.Y, feature/*, hotfix/*
  Pipeline design:
    feature/*: CI only (build + test)
    develop:   CI + deploy to dev environment
    release/*: CI + deploy to staging + quality gates
    main:      production deploy (triggered by merge from release/*)
    hotfix/*:  CI + expedited deploy path

  Jenkins setup: Multi-Branch Pipeline with branch-specific when{}
conditions
  Complexity: Higher — each branch type has different pipeline behavior

RECOMMENDED FOR MOST TEAMS: Trunk-Based Development
  Fewer branches → simpler pipeline → less config drift → faster feedback
```

💬 Short Crisp Interview Answer

> *"Jenkins+Git integration covers three areas: authentication, trigger mechanism, and clone strategy. For auth, SSH keys or GitHub App credentials are preferred over username/password for server-to-server use — GitHub Apps provide auto-rotating short-lived tokens and higher API rate limits. For triggering, webhooks always beat polling: webhooks fire immediately on push with near-zero Controller overhead, while polling adds latency and scales poorly (500 repos × 5-minute polling = 100 API calls/minute). In production I use webhooks as primary trigger with a long-interval `pollSCM('H * * * *')` as fallback in case a webhook delivery fails. For cloning, shallow clone (`depth: 1`) is the default for CI — it fetches only recent history and is dramatically faster for large repos. Sparse checkout layers on top for monorepos — fetching only the subdirectories relevant to the service being built."*

⚠️ Tricky Edge Cases & Gotchas

- **`GIT_BRANCH` contains `origin/main`, `BRANCH_NAME` contains `main`.** `GIT_BRANCH` is set by the Git plugin and includes the remote prefix. `BRANCH_NAME` is set by Multi-Branch Pipeline and is the clean branch name. Use `BRANCH_NAME` for branch conditions; use `GIT_COMMIT` for the actual SHA.
- **Shallow clone breaks `git diff --name-only HEAD~1`.** If you clone with `depth: 1`, there's only one commit in the local history. `HEAD~1` doesn't exist, so the diff command fails. Increase depth to at least 2: `depth: 2`, or use `git diff --name-only origin/main...HEAD` (comparing refs, not local history).
- **`checkout scm` in Multi-Branch Pipeline uses the job's configured SCM.** In a regular Pipeline job, `checkout scm` uses whatever SCM is configured in the job definition. In a Multi-Branch Pipeline, it uses the Multi-Branch job's SCM, checking out the specific branch that triggered the build. This is almost always what you want.
- **Webhook delivery order is not guaranteed.** If two rapid pushes happen within seconds, GitHub may deliver the second webhook before the first build finishes — or deliver them out of order. Use `disableConcurrentBuilds()` with `abortPrevious: true` to handle this.

Topic 6.2 — Jenkins + Docker ⚠️

🟡 Intermediate | Building and Managing Container Images

📌 What It Is — In Simple Terms

Jenkins+Docker integration covers two distinct activities: (1) using Docker containers as the build environment (agents) and (2) building Docker images as part of the pipeline output. Understanding which you're doing, and how to do each correctly in different infrastructure contexts, is a heavily tested topic because the wrong approach creates serious security vulnerabilities.

🔍 The Two Docker Use Cases — Never Confuse Them

```
Use Case A: Docker AS the build environment
  "I want each Jenkins build to run inside a Docker container"
  Tools: Docker Plugin (on Docker hosts) OR Kubernetes Plugin (on K8s)
  Result: Isolated, ephemeral build environment per build
  See also: Category 5.5 (K8s Plugin), 5.6 (Docker Plugin)

Use Case B: Docker IN the build (building images)
  "My pipeline output is a Docker image that I push to a registry"
  Tools: docker CLI, Kaniko, BuildKit — run AS STEPS in the pipeline
  Result: Docker image artifact ready for deployment
  THIS IS WHAT CATEGORY 6.2 IS ABOUT
```

⚙️ Approach 1: Docker CLI on a Docker-Enabled Agent

```
// Agent has Docker installed and is in the docker group
// Straightforward but requires Docker on every build agent

pipeline {
    agent { label 'docker-host' }    // Agent with Docker daemon

    environment {
        REGISTRY   = 'registry.example.com'
        IMAGE_NAME = "${REGISTRY}/myapp"
        IMAGE_TAG  = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build Image') {
            steps {
                sh """
                    docker build \
                        --build-arg BUILD_NUMBER=${env.BUILD_NUMBER} \
                        --build-arg GIT_COMMIT=${env.GIT_COMMIT} \
                        --label "build.number=${env.BUILD_NUMBER}" \
                        --label "git.commit=${env.GIT_COMMIT}" \
                        --label "git.branch=${env.BRANCH_NAME}" \
                        -t ${IMAGE_NAME}:${IMAGE_TAG} \
                        -t ${IMAGE_NAME}:latest \
                        -f Dockerfile \
                        .
                """
            }
        }

        stage('Test Image') {
            steps {
                // Run tests inside the built image
                sh """
                    docker run --rm \
                        -v \$(pwd)/test-results:/app/test-results \
                        -e CI=true \
                        ${IMAGE_NAME}:${IMAGE_TAG} \
                        sh -c 'npm test && cp test-results/*.xml
/app/test-results/'
                """
            }
            post {
                always { junit 'test-results/*.xml' }
                always { sh "docker rmi ${IMAGE_NAME}:${IMAGE_TAG} ||
true" }
            }
        }

        stage('Security Scan') {
            steps {
                sh """
                    # Trivy: scan for OS and library vulnerabilities
```

```
                    docker run --rm \
                        -v /var/run/docker.sock:/var/run/docker.sock \
                        -v \$HOME/.cache/trivy:/root/.cache/trivy \
                        aquasec/trivy:latest image \
                        --exit-code 1 \
                        --severity HIGH,CRITICAL \
                        --format table \
                        ${IMAGE_NAME}:${IMAGE_TAG}
                """
            }
        }

        stage('Push') {
            when {
                anyOf { branch 'main'; branch pattern: 'release/.*',
comparator: 'REGEXP' }
            }
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'registry-credentials',
                    usernameVariable: 'REG_USER',
                    passwordVariable: 'REG_PASS'
                )]) {
                    sh """
                        echo "\$REG_PASS" | docker login ${REGISTRY} -u
"\$REG_USER" --password-stdin
                        docker push ${IMAGE_NAME}:${IMAGE_TAG}
                        # Only push :latest for main branch builds
                        if [ "${env.BRANCH_NAME}" = "main" ]; then
                            docker push ${IMAGE_NAME}:latest
                        fi
                        docker logout ${REGISTRY}
                    """
                }
            }
        }

        stage('Cleanup') {
            steps {
                sh """
                    docker rmi ${IMAGE_NAME}:${IMAGE_TAG} || true
                    docker rmi ${IMAGE_NAME}:latest || true
                    docker image prune -f
                """
            }
        }
    }
}
```

⚙️ Approach 2: Docker Pipeline Plugin (docker.build DSL)

```
// Requires: docker-workflow plugin
// Agent needs Docker installed

pipeline {
    agent { label 'docker-host' }

    stages {
        stage('Build & Push') {
            steps {
                script {
                    // docker.withRegistry handles login/logout
automatically
                    docker.withRegistry('https://registry.example.com',
'registry-credentials') {
                        def image =
docker.build("myapp:${env.BUILD_NUMBER}", '--no-cache .')

                        // Run tests inside the image
                        image.inside('-e CI=true') {
                            sh 'npm test'
                        }

                        // Push both tagged and latest
                        image.push()
                        if (env.BRANCH_NAME == 'main') {
                            image.push('latest')
                        }
                    }
                }
            }
        }
    }
}
```

⚙️ Approach 3: Kaniko — Build Images Without Docker Daemon ✅ Recommended for K8s

```
Why Kaniko:
  - Does NOT require Docker daemon
  - Does NOT require privileged containers
  - Reads Dockerfile, builds image layers, pushes to registry
  - Runs as a normal unprivileged container in Kubernetes pod
  - Supports layer caching via a cache registry

Kaniko vs DinD vs DooD:
  DinD (Docker-in-Docker): requires privileged: true → host root access →
AVOID
  DooD (Docker-outside-of-Docker): mounts host Docker socket → still host
access → AVOID
  Kaniko: no daemon, no privileged, pure container → USE THIS
```

```
// In a Kubernetes pod template with Kaniko container:
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: kaniko
    image: gcr.io/kaniko-project/executor:v1.21.0-debug
    command: ["sleep"]
    args: ["infinity"]
    volumeMounts:
    - name: kaniko-secret
      mountPath: /kaniko/.docker
    resources:
      requests:
        cpu: "500m"
        memory: "1Gi"
      limits:
        cpu: "1"
        memory: "2Gi"
  volumes:
  - name: kaniko-secret
    secret:
      secretName: docker-registry-secret   # kubectl create secret docker-
registry
      items:
      - key: .dockerconfigjson
        path: config.json
'''
        }
    }

    environment {
        REGISTRY   = 'registry.example.com'
        IMAGE_NAME = "${REGISTRY}/myapp"
        IMAGE_TAG  = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build & Push') {
            steps {
                container('kaniko') {
                    sh """
                        /kaniko/executor \
                            --context=dir:///workspace \
                            --dockerfile=/workspace/Dockerfile \
                            --destination=${IMAGE_NAME}:${IMAGE_TAG} \
                            --destination=${IMAGE_NAME}:latest \
```

```
                                --cache=true \
                                --cache-repo=${REGISTRY}/myapp/cache \
                                --build-arg BUILD_NUMBER=${env.BUILD_NUMBER} \
                                --build-arg GIT_COMMIT=${env.GIT_COMMIT} \
                                --label "git.commit=${env.GIT_COMMIT}" \
                                --push-retry 3 \
                                --compressed-caching=false \
                                --snapshot-mode=redo
                        """
                    }
                }
            }
        }
    }
```

⚙️ Multi-Stage Dockerfile Best Practices

```
# ── PRODUCTION-GRADE MULTI-STAGE DOCKERFILE ──────────────────────────

# Stage 1: Build (fat image with build tools)
FROM maven:3.9-eclipse-temurin-17 AS builder

WORKDIR /app

# Cache dependency layer separately (invalidated only when pom.xml
changes)
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Build the application
COPY src/ ./src/
RUN mvn clean package -DskipTests -B

# Stage 2: Test (run tests in the build environment)
FROM builder AS tester
RUN mvn test -B

# Stage 3: Runtime (thin image with just what's needed to run)
FROM eclipse-temurin:17-jre-alpine AS runtime

# Non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

# Copy ONLY the JAR from the builder stage — not Maven, not source code
COPY --from=builder /app/target/*.jar app.jar

# Change ownership
RUN chown -R appuser:appgroup /app
```

```
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
    CMD wget -q -O /dev/null http://localhost:8080/actuator/health || exit
1

EXPOSE 8080

ENTRYPOINT ["java", \
    "-XX:+UseContainerSupport", \
    "-XX:MaxRAMPercentage=75.0", \
    "-jar", "app.jar"]
```

```
// Building multi-stage Dockerfile in pipeline:
// Target specific stage for testing vs final image
stage('Run Unit Tests') {
    steps {
        sh """
            docker build \
                --target tester \
                --build-arg BUILDKIT_INLINE_CACHE=1 \
                -t myapp:test-${env.BUILD_NUMBER} \
                .
            # Extract test results from the container
            docker create --name test-container
myapp:test-${env.BUILD_NUMBER}
            docker cp test-container:/app/target/surefire-reports ./test-
results
            docker rm test-container
        """
    }
    post {
        always {
            junit 'test-results/**/*.xml'
            sh "docker rmi myapp:test-${env.BUILD_NUMBER} || true"
        }
    }
}

stage('Build Production Image') {
    steps {
        sh """
            # Build only the 'runtime' stage (default - builds all stages
up to target)
            DOCKER_BUILDKIT=1 docker build \
                --target runtime \
                --cache-from ${REGISTRY}/myapp:cache \
                --build-arg BUILDKIT_INLINE_CACHE=1 \
                -t ${IMAGE_NAME}:${IMAGE_TAG} \
```

```
                .
        """
    }
}
```

⚙️ Docker Layer Caching in CI

```
Layer caching is critical for build performance.
Without caching, every build downloads all dependencies from scratch.

Strategy 1: Registry-based caching (BuildKit)
  Push: docker buildx build --cache-to
type=registry,ref=registry.io/myapp:cache ...
  Pull: docker buildx build --cache-from
type=registry,ref=registry.io/myapp:cache ...

Strategy 2: Volume-based caching (local Docker host)
  Mount Docker's layer cache into the build:
  -v /var/lib/docker:/var/lib/docker (NOT recommended — host coupling)

Strategy 3: Kaniko cache repository
  --cache=true
  --cache-repo=registry.io/myapp/cache
  Kaniko pushes intermediate layers to the cache repo and pulls on next
build

Strategy 4: BuildKit inline cache
  --build-arg BUILDKIT_INLINE_CACHE=1
  Embeds cache metadata into the pushed image
  Next build: --cache-from the previously pushed image

Performance impact:
  Without caching: Maven build = ~5 minutes (downloading 200MB of deps)
  With caching:    Maven build = ~45 seconds (deps in layer cache)
```

⚙️ Image Tagging Strategy

```
Common tagging strategies and their tradeoffs:

Strategy                Example              Pros                    Cons
:latest               myapp:latest          Simple
Mutable, no traceability
:BUILD_NUMBER         myapp:42              Jenkins-traceable
Resets if build history cleared
:GIT_SHA              myapp:abc1234         Full traceability
Opaque to humans
:VERSION              myapp:1.2.3           Human-readable
```

```
Requires version management
:VERSION-BUILD-SHA    myapp:1.2.3-42-abc    Best traceability        Long
tag names

RECOMMENDED: :VERSION-BUILD_NUMBER-GIT_SHA (7 chars)
  myapp:1.2.3-42-abc1234

  Traceability: commit abc1234 → build #42 → image version 1.2.3
  Human-readable: humans see the version number
  Unique: BUILD_NUMBER prevents collisions if same SHA is rebuilt
  Machine-parseable: structured format enables automation

In Jenkinsfile:
  env.APP_VERSION = sh(script: 'cat VERSION', returnStdout: true).trim()
  env.IMAGE_TAG   =
"${env.APP_VERSION}-${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
  // → 1.2.3-42-abc1234
```

🔑 Key Concepts

Concept	Meaning
DinD	Docker in Docker — full daemon inside container — requires privileged
DooD	Docker outside of Docker — mount host socket — still host access risk
Kaniko	Daemonless image builder — unprivileged, recommended for K8s
Multi-stage build	Multiple FROM stages — build stage (fat) vs runtime stage (thin)
Layer caching	Reuse unchanged layers from previous builds — critical for speed
BuildKit	Next-gen Docker build engine — parallel stages, better caching, secrets
`--cache-from`	Pull cached layers from a registry before building
`BUILDKIT_INLINE_CACHE=1`	Embed cache metadata in pushed image for future `--cache-from`
Image signing	Cosign/Notary — cryptographic proof that an image came from your CI
Ephemeral registry login	`docker login` + build + push + `docker logout` in a single step

---

💬 Short Crisp Interview Answer

> *"Jenkins+Docker integration means using Jenkins to build Docker images as CI artifacts. The approach depends on infrastructure. On a VM-based agent with Docker installed, you use `docker build` directly or the Docker Pipeline Plugin's `docker.withRegistry()` for automatic login/logout. On Kubernetes pod agents — the modern approach — you use Kaniko, which builds images without a Docker daemon and runs unprivileged. DinD and DooD require privileged containers or host Docker socket access, which grants effective root on the node — avoid in production. For Dockerfile design, multi-stage builds are mandatory: a builder stage with all tools, a*

> *separate runtime stage with only the JRE and application JAR. This reduces the runtime image from 800MB to 150MB and removes build tools that are attack surface. Layer caching is critical for build speed: without it, a Maven build downloads 200MB of dependencies every run; with registry-based BuildKit caching, unchanged layers are reused in under a minute."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`docker login` credentials in logs.** Never pass credentials as `docker login -u user -p password` — the password appears in the process list and potentially in logs. Always use `echo "$PASS" | docker login -u "$USER" --password-stdin` or `docker.withRegistry()` which handles this automatically.
- **Concurrent builds on same Docker host share the layer cache.** Two builds of the same image running simultaneously can corrupt each other's layer cache writes. Use unique image tags per build (include BUILD_NUMBER) to avoid tag collisions. The layer cache itself is generally safe for concurrent reads but watch for edge cases on heavy concurrent writes.
- **`:latest` tag is dangerous for production.** If you always push `:latest` and a deployment uses `imagePullPolicy: Always`, every pod restart pulls the newest image — even if you deployed `:1.2.3` intentionally. Always deploy with an immutable tag. Use `:latest` only as a convenience alias for development.
- **Kaniko `--context=dir:///workspace`** — the triple slash and exact path matter. Kaniko reads context from the local filesystem. The Jenkins workspace is mounted at the path that `pwd` returns. Use `dir:///workspace` where `/workspace` is the actual workspace path (or use `$(pwd)` dynamically).
- **`docker image prune` in CI can delete another build's layers.** On shared Docker hosts with many concurrent builds, `docker image prune -f` deletes ALL untagged images — including layers that another concurrent build is using. Use `docker rmi <specific-image>` instead of prune, or use unique image names.

Topic 6.3 — Jenkins + Kubernetes

🔴 Advanced | Deploying to and Running on Kubernetes

📌 What It Is — In Simple Terms

Jenkins+Kubernetes integration has two dimensions: (1) Jenkins running ON Kubernetes with Kubernetes pod agents (covered in Category 5.5), and (2) Jenkins pipelines DEPLOYING TO Kubernetes clusters using `kubectl`, `helm`, and `kustomize`. This topic focuses on the deployment side — how Jenkins pipelines safely and reliably update running services in Kubernetes clusters.

⚙️ Authentication: Jenkins → Kubernetes Cluster

```
Jenkins needs credentials to talk to target Kubernetes clusters.

Method 1: Kubeconfig file credential (simplest)
  - Store kubeconfig as a Jenkins Secret File credential
  - withCredentials([file(credentialsId: 'kubeconfig-staging', variable:
'KUBECONFIG')]){ ... }
  - Standard: any kubectl/helm command works automatically
  - Risk: kubeconfig typically contains long-lived cluster admin
credentials
  - Better: use kubeconfig with RBAC-scoped service account token

Method 2: Service Account Token (fine-grained)
  - Create a Kubernetes ServiceAccount with RBAC Role limited to what
Jenkins needs
  - Store the SA token as Jenkins Secret Text credential
  - Pass as --token flag or set in kubeconfig

Method 3: In-cluster config (for Jenkins running in K8s)
  - Jenkins pod runs with a ServiceAccount
  - Kubernetes SDK uses in-cluster config automatically
  - Most secure: no external credential management

Method 4: Cloud IAM (AWS EKS/GCP GKE/Azure AKS)
  - EKS: Jenkins IAM role maps to Kubernetes RBAC via aws-auth configmap
  - GKE: Workload Identity maps GCP SA to K8s SA
  - AKS: Pod Identity/Workload Identity maps Azure Managed Identity to K8s
SA
  - Best practice for cloud-hosted clusters: no long-lived tokens
```

```
# Create minimal RBAC for Jenkins deploy service account:
cat <<EOF | kubectl apply -f -
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-deployer
  namespace: jenkins
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: jenkins-deployer-role
rules:
  # Helm needs these for release management
  - apiGroups: [""]
    resources: ["namespaces", "configmaps", "secrets", "services",
                "pods", "persistentvolumeclaims", "serviceaccounts"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets", "statefulsets",
"daemonsets"]
```

```
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: ["networking.k8s.io"]
    resources: ["ingresses"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  - apiGroups: ["batch"]
    resources: ["jobs", "cronjobs"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: jenkins-deployer-binding
subjects:
  - kind: ServiceAccount
    name: jenkins-deployer
    namespace: jenkins
roleRef:
  kind: ClusterRole
  name: jenkins-deployer-role
  apiGroup: rbac.authorization.k8s.io
EOF
```

⚙️ Helm Deployment Pattern — Production Complete

```
// Complete production Helm deployment pipeline

pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: helm
    image: alpine/helm:3.14.0
    command: ["cat"]
    tty: true
    resources:
      requests: {cpu: "100m", memory: "128Mi"}
      limits:   {cpu: "200m", memory: "256Mi"}
  - name: kubectl
    image: bitnami/kubectl:1.29
    command: ["cat"]
    tty: true
    resources:
      requests: {cpu: "100m", memory: "128Mi"}
      limits:   {cpu: "200m", memory: "256Mi"}
'''
        }
```

```
    }

    environment {
        APP_NAME  = 'myservice'
        REGISTRY  = 'registry.example.com'
        IMAGE_TAG = "${env.BUILD_NUMBER}-${env.GIT_COMMIT?.take(7)}"
    }

    stages {
        stage('Deploy: Staging') {
            when { branch 'main' }
            steps {
                container('helm') {
                    withCredentials([file(credentialsId: 'kubeconfig-
staging',
                                          variable: 'KUBECONFIG')]) {
                        sh """
                            helm upgrade --install ${APP_NAME} ./chart \
                                --namespace staging \
                                --create-namespace \
                                --set
image.repository=${REGISTRY}/${APP_NAME} \
                                --set image.tag=${IMAGE_TAG} \
                                --set replicaCount=2 \
                                -f chart/values-staging.yaml \
                                --atomic \
                                --wait \
                                --timeout=5m \
                                --history-max=5
                        """
                    }
                }
            }
            post {
                success {
                    container('kubectl') {
                        withCredentials([file(credentialsId: 'kubeconfig-
staging',
                                              variable: 'KUBECONFIG')]) {
                            sh """
                                kubectl rollout status
deployment/${APP_NAME} \
                                    -n staging --timeout=3m
                                kubectl get pods -n staging -l
app=${APP_NAME}
                            """
                        }
                    }
                }
                failure {
                    container('helm') {
                        withCredentials([file(credentialsId: 'kubeconfig-
staging',
                                              variable: 'KUBECONFIG')]) {
```

```
                            sh "helm history ${APP_NAME} -n staging --max
3 || true"
                        }
                    }
                }
            }
        }

        stage('Smoke Tests') {
            when { branch 'main' }
            agent { label 'test' }
            steps {
                sh """
                    # Wait for endpoint to be reachable
                    for i in \$(seq 1 12); do
                        STATUS=\$(curl -s -o /dev/null -w '%{http_code}' \
                                https://staging.example.com/health)
                        if [ "\$STATUS" = "200" ]; then
                            echo "Service healthy ✅"
                            exit 0
                        fi
                        echo "Attempt \$i: status=\$STATUS — retrying in
10s"
                        sleep 10
                    done
                    echo "Service not healthy after 120s ❌"
                    exit 1
                """
            }
        }

        stage('Deploy: Production') {
            when { branch 'main' }
            steps {
                // Manual approval gate
                input(
                    message:   "Deploy ${APP_NAME}:${IMAGE_TAG} to
PRODUCTION?",
                    ok:        'Deploy',
                    submitter: 'release-managers',
                    submitterParameter: 'DEPLOYER'
                )

                container('helm') {
                    withCredentials([file(credentialsId: 'kubeconfig-
production',
                                          variable: 'KUBECONFIG')]) {
                        sh """
                            # Rolling update with maxSurge/maxUnavailable
from chart values
                            helm upgrade --install ${APP_NAME} ./chart \
                                --namespace production \
                                --set
image.repository=${REGISTRY}/${APP_NAME} \
```

```
                                --set image.tag=${IMAGE_TAG} \
                                -f chart/values-production.yaml \
                                --atomic \
                                --wait \
                                --timeout=10m \
                                --history-max=10
                        """
                    }
                }
            }
            post {
                success {
                    echo "Deployed to production by ${env.DEPLOYER}"
                    // Record deployment event for DORA metrics
                    sh """
                        curl -X POST
https://deployments.internal/api/record \
                            -H 'Content-Type: application/json' \
                            -d
'{"service":"${APP_NAME}","version":"${IMAGE_TAG}",

"env":"production","deployer":"${env.DEPLOYER ?: "auto"}",
                                "timestamp":"${new Date().format("yyyy-
MM-dd'T'HH:mm:ss'Z'")}"}'
                    """
                }
                failure {
                    container('helm') {
                        withCredentials([file(credentialsId: 'kubeconfig-
production',
                                              variable: 'KUBECONFIG')]) {
                            // --atomic auto-rolled back; show the
rollback state
                            sh """
                                echo "Deploy failed — helm --atomic
triggered rollback"
                                helm history ${APP_NAME} -n production --
max 5
                            """
                        }
                    }
                }
            }
        }
    }
}
```

⚙️ Canary Deployment Pattern in Jenkins

```
// Canary: deploy to 10% of traffic first, observe, then full rollout

stage('Canary Deploy') {
    steps {
        container('kubectl') {
            withCredentials([file(credentialsId: 'kubeconfig-production',
variable: 'KUBECONFIG')]) {
                sh """
                    # Deploy canary with 10% weight using Argo Rollouts or
Istio
                    # Example: Argo Rollouts approach
                    kubectl argo rollouts set image myapp-rollout \
                        myapp=${REGISTRY}/${APP_NAME}:${IMAGE_TAG} \
                        -n production

                    # Pause rollout at 10% canary step
                    # (step defined in Rollout spec - Jenkins just
promotes)
                """
            }
        }
    }
}

stage('Monitor Canary') {
    steps {
        script {
            // Monitor for 5 minutes
            sleep(time: 5, unit: 'MINUTES')

            // Check error rate (query Prometheus/Datadog)
            withCredentials([string(credentialsId: 'prometheus-token',
variable: 'PROM_TOKEN')]) {
                def errorRate = sh(
                    script: """
                        curl -s -G \
                            --data-urlencode
'query=sum(rate(http_requests_total{app="myapp",status=~"5..",env="canary"
}[5m])) / sum(rate(http_requests_total{app="myapp",env="canary"}[5m])) *
100' \
                            https://prometheus.example.com/api/v1/query \
                            -H "Authorization: Bearer \$PROM_TOKEN" \
                            | python3 -c "import sys,json;
d=json.load(sys.stdin); print(d['data']['result'][0]['value'][1] if
d['data']['result'] else '0')"
                    """,
                    returnStdout: true
                ).trim().toDouble()

                if (errorRate > 1.0) {
                    // Abort canary - rollback
                    sh 'kubectl argo rollouts abort myapp-rollout -n
production'
```

```
                    error("Canary error rate ${errorRate}% exceeds 1%
threshold — rollback triggered")
                }
                echo "Canary healthy — error rate: ${errorRate}% ✅"
            }
        }
    }
}

stage('Promote Canary') {
    steps {
        container('kubectl') {
            withCredentials([file(credentialsId: 'kubeconfig-production',
variable: 'KUBECONFIG')]) {
                sh 'kubectl argo rollouts promote myapp-rollout -n
production --full'
            }
        }
    }
}
```

⚙️ Kustomize Deployment Pattern

```
// Kustomize: overlay-based configuration management (alternative to Helm)

stage('Deploy with Kustomize') {
    steps {
        container('kubectl') {
            withCredentials([file(credentialsId: 'kubeconfig-staging',
variable: 'KUBECONFIG')]) {
                sh """
                    # Update the image tag in kustomization.yaml
                    cd k8s/overlays/staging
                    kustomize edit set image
myapp=${REGISTRY}/${APP_NAME}:${IMAGE_TAG}

                    # Apply with server-side apply (handles large
resources)
                    kubectl apply -k . \
                        --server-side \
                        --force-conflicts \
                        -n staging

                    # Wait for rollout
                    kubectl rollout status deployment/${APP_NAME} \
                        -n staging \
                        --timeout=5m
                """
            }
        }
```

```
    }
}
```

🔑 Key Concepts

Concept	Meaning
`helm upgrade --install`	Create if not exists, upgrade if exists — idempotent deploy
`--atomic`	Helm auto-rollback to previous release on deploy failure
`--wait`	Helm blocks until all K8s resources are ready
`--history-max`	Limit stored Helm release history (prevent unbounded growth)
Kubeconfig credential	Jenkins Secret File containing kubeconfig for target cluster
RBAC service account	Least-privilege K8s identity for Jenkins to deploy with
`kubectl rollout status`	Verify deployment rolled out successfully
Canary deployment	Route small % of traffic to new version, observe, then promote
Argo Rollouts	Kubernetes controller for advanced deploy strategies (canary, blue-green)
GitOps	Promotion = commit to config repo; Argo CD/Flux applies to cluster

---

💬 Short Crisp Interview Answer

> *"Jenkins+Kubernetes has two dimensions: Jenkins running on K8s with pod agents (Category 5.5) and Jenkins pipelines deploying to K8s clusters. For deployments, I use Helm inside a kubectl/helm container in the build pod. Authentication is via a kubeconfig stored as a Jenkins Secret File credential, injected with `withCredentials([file(...)])`. The kubeconfig references a service account with a minimal RBAC role — not cluster-admin. Key Helm flags: `--atomic` auto-rolls back on failure, `--wait` blocks until all pods are ready, `--history-max` caps release history size. The deploy pipeline stages: build image → deploy to staging → smoke tests → manual approval gate → production deploy. For zero-downtime, `--atomic` with Helm's rolling update strategy, or Argo Rollouts for canary with metric-based promotion. The modern alternative is GitOps: Jenkins commits the new image tag to the config repo and Argo CD handles the cluster apply — Jenkins never touches the cluster directly, which is a strong security boundary."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`helm upgrade --atomic` rolls back but doesn't fail the Jenkins stage by default.** `--atomic` rolls back to the previous release when the upgrade fails, but the `helm` process still exits with a non-zero code — Jenkins marks the stage as failed. This is correct behavior, but operators sometimes

expect "rolled back = safe = green." Always alert on failed production deploys even if Helm auto-rolled back.

- **Kubeconfig with `cluster-admin` is a security anti-pattern.** Many tutorials use `cluster-admin` for Jenkins. In production, scope the service account's RBAC Role to the specific namespaces and resources Jenkins needs. A compromised Jenkins shouldn't be able to delete all cluster resources.
- **`kubectl wait` vs `kubectl rollout status`.** `kubectl wait pod --for=condition=Ready` waits for individual pods to be ready. `kubectl rollout status deployment/myapp` waits for the entire rolling update to complete (all replicas updated and ready). Use `rollout status` for deployments.
- **Helm `--wait` timeout.** If `--wait --timeout=5m` expires before pods become ready, Helm exits with an error and `--atomic` triggers a rollback. Set `--timeout` generously enough for slow pod startup (especially JVM services with 60-90 second startup times).

Topic 6.4 — Jenkins + SonarQube

🟡 Intermediate | Code Quality as a Pipeline Gate

📌 What It Is — In Simple Terms

SonarQube is a code quality and security analysis platform that inspects source code for bugs, code smells, security vulnerabilities, and technical debt. The Jenkins+SonarQube integration runs analysis during the pipeline and — critically — gates the build on the SonarQube Quality Gate result. Code that fails the Quality Gate blocks the pipeline, preventing low-quality code from proceeding to deployment.

🔍 Why Quality Gates Matter

```
Without quality gate:
  Developer pushes code → Jenkins builds → SonarQube runs → findings
reported
  → Developer ignores the SonarQube report → bad code reaches production
  → Technical debt accumulates → velocity decreases → bugs increase

With quality gate as pipeline blocker:
  Developer pushes code → Jenkins builds → SonarQube runs → 5 critical
issues found
  → Quality Gate: FAILED → Pipeline: FAILED → Code cannot merge/deploy
  → Developer must fix the issues → quality maintained
  → Clean code reaches production consistently
```

⚙️ Setup: Jenkins + SonarQube Connection

```
Prerequisites:
  1. SonarQube server running (https://sonarqube.example.com)
  2. SonarQube token: SQ Admin → Security → User Tokens → Generate
  3. Jenkins plugin: SonarQube Scanner (sonar plugin)
  4. Jenkins credential: Secret Text with SonarQube token

JCasC configuration:
```

```yaml
# JCasC — SonarQube server configuration
unclassified:
  sonarGlobalConfiguration:
    installations:
      - name: "SonarQube"
        serverUrl: "https://sonarqube.example.com"
        credentialsId: "sonarqube-token"      # Jenkins Secret Text
credential
        mojoVersion: ""
        triggers:
          skipScmCause: false
          skipUpstreamCause: false
        additionalProperties: ""
```

⚙️ Complete SonarQube Pipeline Integration

```
// Full SonarQube integration with quality gate

pipeline {
    agent { label 'java' }

    options {
        timeout(time: 30, unit: 'MINUTES')
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }

        stage('Unit Tests') {
            steps {
                sh 'mvn test'
            }
            post {
                always { junit 'target/surefire-reports/*.xml' }
            }
```

```
        }

        // ── SONARQUBE ANALYSIS ──────────────────────────────────────
        stage('SonarQube Analysis') {
            // Only analyze branches that matter
            when {
                anyOf {
                    branch 'main'
                    branch pattern: 'release/.*', comparator: 'REGEXP'
                    changeRequest()   // Analyze all PRs
                }
            }
            steps {
                // withSonarQubeEnv injects SONAR_HOST_URL and
SONAR_AUTH_TOKEN
                // into the environment for any sonar-scanner call
                withSonarQubeEnv('SonarQube') {
                    sh """
                        mvn sonar:sonar \
                            -Dsonar.projectKey=myapp \
                            -Dsonar.projectName="My Application" \
                            -Dsonar.sources=src/main/java \
                            -Dsonar.tests=src/test/java \
                            -Dsonar.java.binaries=target/classes \
                            -Dsonar.java.test.binaries=target/test-classes
\
                            -
Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml \
                            -Dsonar.junit.reportPaths=target/surefire-
reports \
                            -Dsonar.branch.name=${env.BRANCH_NAME ?:
'main'} \
                            -B
                    """
                    // For PR analysis (adds decoration on the PR in
GitHub/GitLab):
                    // -Dsonar.pullrequest.key=${env.CHANGE_ID} \
                    // -Dsonar.pullrequest.branch=${env.BRANCH_NAME} \
                    // -Dsonar.pullrequest.base=${env.CHANGE_TARGET} \
                }
            }
        }

        // ── QUALITY GATE CHECK ──────────────────────────────────────
        stage('Quality Gate') {
            when {
                anyOf {
                    branch 'main'
                    branch pattern: 'release/.*', comparator: 'REGEXP'
                    changeRequest()
                }
            }
            steps {
                // Wait for SonarQube to finish analysis (async)
```

```
                // timeout: how long to wait for SQ to compute the quality
gate
                timeout(time: 10, unit: 'MINUTES') {
                    // waitForQualityGate polls SonarQube until analysis
is complete
                    // abortPipeline: true → pipeline fails if QG fails
                    def qg = waitForQualityGate(abortPipeline: false)
                    // abortPipeline: false lets us handle failure
ourselves:
                    if (qg.status != 'OK') {
                        // Get details for the failure message
                        echo "Quality Gate FAILED: ${qg.status}"
                        echo "SonarQube dashboard:
${env.SONAR_HOST_URL}/dashboard?id=myapp"
                        currentBuild.description = "QG Failed:
${qg.status}"

                        // Fail the build
                        error("Quality Gate failed: ${qg.status}")
                    }
                    echo "Quality Gate PASSED ✅"
                }
            }
        }

        stage('Deploy') {
            when { branch 'main' }
            steps {
                sh 'helm upgrade --install myapp ./chart --namespace
staging'
            }
        }
    }

    post {
        failure {
            script {
                // Add SonarQube link to Slack failure notification
                def sonarUrl = "${env.SONAR_HOST_URL}/dashboard?id=myapp"
                slackSend(
                    channel: '#build-failures',
                    color: 'danger',
                    message: "❌ Build failed: ${env.JOB_NAME}\n" +
                             "SonarQube: ${sonarUrl}\n" +
                             "Build: ${env.BUILD_URL}"
                )
            }
        }
    }
}
```

⚙️ SonarQube for Non-Java Projects

```
// Node.js / TypeScript:
withSonarQubeEnv('SonarQube') {
    sh """
        # Install sonar-scanner CLI (or use sonarqube-scanner npm package)
        npx sonar-scanner \
            -Dsonar.projectKey=myapp-frontend \
            -Dsonar.sources=src \
            -Dsonar.exclusions='**/*.test.ts,**/node_modules/**' \
            -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info \
            -Dsonar.testExecutionReportPaths=test-report.xml
    """
}

// Python:
withSonarQubeEnv('SonarQube') {
    sh """
        sonar-scanner \
            -Dsonar.projectKey=myapp-python \
            -Dsonar.sources=src \
            -Dsonar.python.coverage.reportPaths=coverage.xml \
            -Dsonar.python.xunit.reportPath=test-results.xml
    """
}

// Go:
withSonarQubeEnv('SonarQube') {
    sh """
        sonar-scanner \
            -Dsonar.projectKey=myapp-go \
            -Dsonar.sources=. \
            -Dsonar.exclusions='**/*_test.go' \
            -Dsonar.tests=. \
            -Dsonar.test.inclusions='**/*_test.go' \
            -Dsonar.go.coverage.reportPaths=coverage.out
    """
}

// Kubernetes-based: use sonar-scanner container in pod template
// Add to pod template YAML:
// - name: sonar-scanner
//   image: sonarsource/sonar-scanner-cli:5.0
//   command: ["cat"]
//   tty: true
// Then: container('sonar-scanner') { sh 'sonar-scanner ...' }
```

⚙️ sonar-project.properties — Version-Controlled Config

```
# sonar-project.properties — checked in at repo root
# Removes the need for long -D flags in Jenkinsfile

sonar.projectKey=myapp
sonar.projectName=My Application
sonar.projectVersion=1.0

# Source and test directories
sonar.sources=src/main/java
sonar.tests=src/test/java

# Compiled classes
sonar.java.binaries=target/classes
sonar.java.test.binaries=target/test-classes

# Test reports
sonar.junit.reportPaths=target/surefire-reports

# Code coverage (must run JaCoCo before sonar:sonar)
sonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml

# Exclusions (don't analyze generated code, vendor code)
sonar.exclusions=**/generated/**,**/vendor/**,**/*.min.js,**/node_modules/
**

# Duplication detection minimum token count
sonar.cpd.java.minimumTokens=50

# Quality gate (referenced by name or key — configured in SonarQube UI)
# sonar.qualitygate.wait=true  # Alternative to waitForQualityGate step
```

⚙️ Quality Gate Configuration in SonarQube

```
SonarQube Quality Gate (configured in SonarQube UI or API):

A Quality Gate is a set of conditions that must all be true for the gate
to pass.

Example production Quality Gate "High Standard":
   ┌──────────────────────────────────────────────────┐
   │ Condition                          | Error threshold  |
   ├──────────────────────────────────────────────────┤
   │ Coverage on New Code               | < 80%            |
   │ Duplicated Lines on New Code       | > 3%             |
   │ New Bugs                           | > 0              |
   │ New Vulnerabilities                | > 0              |
   │ New Security Hotspots Reviewed     | < 100%           |
   │ New Code Smells (severity: BLOCKER)| > 0              |
   └──────────────────────────────────────────────────┘
```

```
"On New Code" = only checks lines changed since last analysis
This prevents legacy code from blocking new development
while still requiring new code to meet standards.
```

🔑 Key Concepts

Concept	Meaning
`withSonarQubeEnv()`	Injects SonarQube URL + token as env vars for scanner
`waitForQualityGate()`	Polls SonarQube until analysis completes, checks QG status
Quality Gate	Set of pass/fail conditions on code metrics
`abortPipeline: true`	Auto-fail pipeline if QG fails
New Code	Lines changed since last analysis — QG can focus only on new code
`sonar-project.properties`	File-based configuration — version-controlled with the code
Branch analysis	SonarQube analyzes different branches separately (SonarQube Developer edition+)
PR decoration	SonarQube posts findings as comments/checks on GitHub/GitLab PRs
Security Hotspots	Code patterns that require manual review — not necessarily bugs
Coverage	% of code exercised by tests — SonarQube reads JaCoCo/Istanbul reports

---

💬 Short Crisp Interview Answer

> *"The Jenkins+SonarQube integration has two steps: analysis and quality gate. Analysis: `withSonarQubeEnv('SonarQube')` injects the server URL and auth token, then `mvn sonar:sonar` (or sonar-scanner CLI) runs the static analysis and uploads results asynchronously. Quality gate: `waitForQualityGate()` polls SonarQube until analysis completes and returns the gate status — PASSED or FAILED. With `abortPipeline: true`, any gate failure fails the build, blocking deployment. The gate is configured in SonarQube with thresholds on metrics like 'coverage on new code must be ≥ 80%' and 'zero new vulnerabilities.' The 'new code' focus is critical: it prevents blocking the entire CI because of legacy technical debt, while ensuring all new code meets standards. Config best practice: check `sonar-project.properties` into the repo so the analysis configuration is version-controlled with the code."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Analysis is ASYNCHRONOUS.** `mvn sonar:sonar` submits the analysis to SonarQube but doesn't wait for it to complete. You MUST call `waitForQualityGate()` in a SEPARATE stage after the analysis stage. Calling it in the same step as the analysis often races — the result isn't ready yet.

- **Coverage must be generated BEFORE sonar:sonar.** SonarQube reads JaCoCo XML from `target/site/jacoco/jacoco.xml` — this file is created by `mvn verify` or `mvn jacoco:report`, not by `mvn sonar:sonar`. If you skip tests or don't generate coverage, SonarQube sees 0% coverage.
- **`waitForQualityGate()` requires the SonarQube webhook.** SonarQube must call back to Jenkins when analysis is complete. Configure this in SonarQube: Administration → Configuration → Webhooks → Add URL: `https://jenkins.example.com/sonarqube-webhook/`. Without this, `waitForQualityGate()` hangs until timeout.
- **Branch analysis requires SonarQube Developer Edition or higher.** Community Edition only analyzes the main branch. Feature branch and PR analysis require Developer Edition. With Community Edition, all branches submit to the same project key — branch metrics are mixed.
- **Quality Gate timeout.** SonarQube analysis can take 5-15 minutes for large codebases. The `timeout()` wrapping `waitForQualityGate()` must be generous enough. A 5-minute timeout that fires before SonarQube finishes analysis causes false failures.

Topic 6.5 — Jenkins + Nexus/Artifactory

🟡 Intermediate | Artifact Storage and Dependency Management

📌 What It Is — In Simple Terms

Nexus Repository and Artifactory are **artifact repository managers** — centralized storage for build outputs (JARs, Docker images, npm packages, Helm charts, Python wheels) and proxies for external dependencies (Maven Central, npmjs.com, PyPI). The Jenkins integration publishes build artifacts to the repo manager and configures builds to resolve dependencies through it rather than directly from the internet.

🔍 Why Artifact Repository Managers Exist

```
Without a repository manager:
  Every build downloads dependencies from the internet:
    Maven build → 200MB from Maven Central → every build → slow
    npm build   → node_modules from npmjs.com → every build
    Multiple teams → same downloads → same network load

  Build outputs are Jenkins artifacts only:
    JAR in Jenkins build #42 → only accessible via Jenkins UI
    If Jenkins history deleted → artifact lost
    Other teams can't find/use the JAR without Jenkins access

With Nexus/Artifactory:
  Proxy: cache external dependencies locally
    Maven build → 200MB from local Nexus → 10x faster → no internet needed

  Hosted: store your own artifacts
    jenkins → publishes myapp-1.2.3.jar to Nexus → versioned, searchable
```

```
    Other teams → resolve myapp:1.2.3 from Nexus in their builds

  Group: expose proxy + hosted as single URL
    Developers configure one URL for all dependencies
```

⚙️ Publishing Artifacts to Nexus — Maven

```
pipeline {
    agent { label 'java' }

    environment {
        NEXUS_URL     = 'https://nexus.example.com'
        NEXUS_REPO    = 'maven-releases'
        NEXUS_REPO_SNAPSHOTS = 'maven-snapshots'
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Publish to Nexus') {
            when {
                anyOf { branch 'main'; branch pattern: 'release/.*',
comparator: 'REGEXP' }
            }
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'nexus-credentials',
                    usernameVariable: 'NEXUS_USER',
                    passwordVariable: 'NEXUS_PASS'
                )]) {
                    // Method 1: Maven deploy (reads
distributionManagement from pom.xml)
                    sh """
                        mvn deploy \
                            -DskipTests \
                            -Dmaven.deploy.skip=false \
                            -
DaltDeploymentRepository=nexus::default::${NEXUS_URL}/repository/${NEXUS_R
EPO} \
                            --settings settings.xml
```

```
                    """
                    // Note: settings.xml contains Nexus credentials in
<server> block
                    // Generated from Jenkins credentials to avoid
hardcoding

                    // Method 2: Nexus Publishing Plugin (Jenkins plugin:
nexus-artifact-uploader)
                    // nexusArtifactUploader(
                    //     nexusVersion:   'nexus3',
                    //     protocol:       'https',
                    //     nexusUrl:       'nexus.example.com',
                    //     groupId:        'com.example',
                    //     version:        sh(script:'mvn help:evaluate -
Dexpression=project.version -q -DforceStdout', returnStdout:true).trim(),
                    //     repository:     env.BRANCH_NAME == 'main' ?
NEXUS_REPO : NEXUS_REPO_SNAPSHOTS,
                    //     credentialsId:  'nexus-credentials',
                    //     artifacts: [
                    //         [artifactId: 'myapp', classifier: '', file:
'target/myapp.jar', type: 'jar'],
                    //         [artifactId: 'myapp', classifier: '', file:
'pom.xml', type: 'pom']
                    //     ]
                    // )
                }
            }
        }

        stage('Publish Docker Image to Nexus') {
            when { branch 'main' }
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'nexus-credentials',
                    usernameVariable: 'NEXUS_USER',
                    passwordVariable: 'NEXUS_PASS'
                )]) {
                    sh """
                        echo "\$NEXUS_PASS" | docker login ${NEXUS_URL} \
                            -u "\$NEXUS_USER" --password-stdin
                        docker tag myapp:${env.BUILD_NUMBER} \
                            ${NEXUS_URL}/myapp:${env.BUILD_NUMBER}
                        docker push ${NEXUS_URL}/myapp:${env.BUILD_NUMBER}
                        docker logout ${NEXUS_URL}
                    """
                }
            }
        }
    }
}
```

⚙️ Publishing to Artifactory — JFrog CLI

```
pipeline {
    agent { label 'java' }

    stages {
        stage('Configure Artifactory') {
            steps {
                // JFrog plugin: rtServer, rtMavenResolver,
rtMavenDeployer
                rtServer(
                    id:          'ARTIFACTORY',
                    url:         'https://mycompany.jfrog.io/artifactory',
                    credentialsId: 'artifactory-credentials'
                )

                // Configure Maven to resolve from and deploy to
Artifactory
                rtMavenResolver(
                    id:             'MAVEN_RESOLVER',
                    serverId:       'ARTIFACTORY',
                    releaseRepo:    'libs-release',
                    snapshotRepo:   'libs-snapshot'
                )

                rtMavenDeployer(
                    id:             'MAVEN_DEPLOYER',
                    serverId:       'ARTIFACTORY',
                    releaseRepo:    'libs-release-local',
                    snapshotRepo:   'libs-snapshot-local',
                    // Add build properties to published artifacts:
                    properties:     ['build.number': env.BUILD_NUMBER,
                                     'git.commit':   env.GIT_COMMIT,
                                     'branch':        env.BRANCH_NAME]
                )
            }
        }

        stage('Build & Deploy') {
            steps {
                // rtMavenRun: Maven run tracked by Artifactory (includes
build info)
                rtMavenRun(
                    tool:       'Maven-3.9',    // Jenkins tool name
                    pom:        'pom.xml',
                    goals:      'clean deploy',
                    resolverId: 'MAVEN_RESOLVER',
                    deployerId: 'MAVEN_DEPLOYER'
                )
            }
        }
```

```
        stage('Publish Build Info') {
            steps {
                // Publish build metadata to Artifactory (enables
traceability)
                rtPublishBuildInfo(serverId: 'ARTIFACTORY')
                // Now in Artifactory: build "myapp" #42 with:
                //   - Published artifacts + checksums
                //   - Dependencies used
                //   - Git commit, branch
                //   - Test results
                //   - Jenkins build URL
            }
        }
    }
}
```

⚙️ Using Nexus/Artifactory as Dependency Proxy

```xml
<!-- settings.xml - tells Maven to use Nexus as mirror for all
dependencies -->
<!-- Generated and placed in workspace by Jenkins pipeline -->
<settings>
  <mirrors>
    <mirror>
      <!-- Proxy ALL external repositories through Nexus -->
      <id>nexus-proxy</id>
      <mirrorOf>*</mirrorOf>
      <url>https://nexus.example.com/repository/maven-group/</url>
    </mirror>
  </mirrors>
  <servers>
    <server>
      <id>nexus-proxy</id>
      <username>${env.NEXUS_USER}</username>
      <password>${env.NEXUS_PASS}</password>
    </server>
  </servers>
  <profiles>
    <profile>
      <id>nexus</id>
      <repositories>
        <repository>
          <id>nexus</id>
          <url>https://nexus.example.com/repository/maven-group/</url>
          <releases><enabled>true</enabled></releases>
          <snapshots><enabled>true</enabled></snapshots>
        </repository>
      </repositories>
    </profile>
  </profiles>
```

```
  <activeProfiles><activeProfile>nexus</activeProfile></activeProfiles>
</settings>
```

```
// Generate settings.xml from Jenkins credentials at build time
// Never check settings.xml with credentials into Git

stage('Configure Maven') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'nexus-credentials',
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASS'
        )]) {
            writeFile file: 'settings.xml', text: """
<settings>
  <servers>
    <server>
      <id>nexus</id>
      <username>\${NEXUS_USER}</username>
      <password>\${NEXUS_PASS}</password>
    </server>
  </servers>
  <mirrors>
    <mirror>
      <id>nexus</id>
      <mirrorOf>*</mirrorOf>
      <url>${env.NEXUS_URL}/repository/maven-group/</url>
    </mirror>
  </mirrors>
</settings>"""
        }
        sh 'mvn -s settings.xml clean package'
    }
}
```

⚙️ npm / PyPI / Helm Chart Publishing

```
// npm: publish to Nexus npm hosted repository
stage('Publish npm Package') {
    steps {
        withCredentials([string(credentialsId: 'nexus-npm-token',
variable: 'NPM_TOKEN')]) {
            sh """
                # Configure npm to use Nexus registry
                echo "registry=https://nexus.example.com/repository/npm-
hosted/" > .npmrc
                echo "//nexus.example.com/repository/npm-
hosted/:_authToken=\${NPM_TOKEN}" >> .npmrc
```

```
                npm publish
                rm -f .npmrc   # Clean up credential file
            """
        }
    }
}

// Python: publish wheel to Nexus PyPI repository
stage('Publish Python Package') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'nexus-credentials',
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASS'
        )]) {
            sh """
                pip install twine --break-system-packages
                python setup.py sdist bdist_wheel
                twine upload \
                    --repository-url
https://nexus.example.com/repository/pypi-hosted/ \
                    --username "\$NEXUS_USER" \
                    --password "\$NEXUS_PASS" \
                    dist/*
            """
        }
    }
}

// Helm chart: push to Nexus Helm repository
stage('Publish Helm Chart') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'nexus-credentials',
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASS'
        )]) {
            sh """
                helm package ./chart --version ${env.APP_VERSION}
                curl -u "\$NEXUS_USER:\$NEXUS_PASS" \
                    --upload-file myapp-${env.APP_VERSION}.tgz \
                    https://nexus.example.com/repository/helm-hosted/
            """
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning

Concept	Meaning
Hosted repository	Stores your own artifacts (build outputs)
Proxy repository	Caches external dependencies locally (Maven Central, npmjs.com)
Group repository	Virtual repo that combines hosted + proxy — single URL for builds
Build info	Artifactory's metadata: links artifact → build → git commit → test results
`rtPublishBuildInfo`	JFrog plugin step: publishes build metadata to Artifactory
settings.xml	Maven config file — specifies where to resolve/deploy; never commit with credentials
Checksum verification	Nexus/Artifactory verify artifact integrity via SHA1/MD5/SHA256
Snapshot vs Release	Snapshots = mutable dev versions (1.2.3-SNAPSHOT); Releases = immutable
`.npmrc`	npm config file — specifies registry; generate from credentials, clean up after

---

💬 Short Crisp Interview Answer

> *"Nexus and Artifactory serve two purposes: dependency proxying and artifact publishing. As a proxy, they cache external packages (Maven Central, npmjs.com, PyPI) locally — builds resolve from the internal Nexus rather than the internet, which is faster, more reliable, and works in air-gapped environments. As a hosted repository, they store your build outputs — JARs, Docker images, Helm charts — with version history and traceability. The Jenkins integration for Maven uses `mvn deploy` pointed at Nexus, with settings.xml generated at build time from Jenkins credentials (never committed to Git with actual credentials). For Artifactory specifically, the JFrog plugin provides `rtMavenRun` + `rtPublishBuildInfo` which publishes rich build metadata — linking the artifact to the exact git commit, test results, and dependencies used — enabling complete artifact traceability."*

⚠ Tricky Edge Cases & Gotchas

- **Never commit `settings.xml` with credentials to Git.** The settings file contains Nexus username/password. Generate it dynamically from Jenkins credentials using `writeFile` and clean it up in `post { always { sh 'rm -f settings.xml' } }`.
- **SNAPSHOT vs RELEASE repository targeting.** Maven automatically routes based on version: `1.2.3-SNAPSHOT` goes to the snapshot repo, `1.2.3` goes to the release repo. Both must be configured. Deploying a SNAPSHOT to a release repo fails. Deploying a RELEASE to a snapshot repo succeeds but is semantically wrong (releases should be immutable).
- **Artifactory build info requires rtMavenRun — not just `sh 'mvn deploy'`.** If you call `sh 'mvn deploy'` instead of `rtMavenRun`, Artifactory receives the artifact but not the build metadata. You lose the build→artifact traceability chain.
- **npm `.npmrc` cleanup.** If the build fails before cleaning up `.npmrc`, the credential file remains in the workspace. Add `sh 'rm -f .npmrc'` in `post { always {} }` to ensure cleanup regardless of build result.

Topic 6.6 — Jenkins + Vault/Secrets Managers

🔴 Advanced | Dynamic Secrets and Zero Long-Lived Credentials

📌 What It Is — In Simple Terms

HashiCorp Vault (and cloud equivalents: AWS Secrets Manager, GCP Secret Manager, Azure Key Vault) are dedicated secrets management systems. Unlike Jenkins' built-in credential store, Vault provides dynamic secrets (credentials generated on-demand with a TTL, automatically revoked after use), secret leasing/rotation, and full audit trails of every secret access. The Jenkins+Vault integration enables pipelines to get secrets directly from Vault rather than from Jenkins' credential store — eliminating long-lived static credentials in Jenkins entirely.

🔍 Static Credentials vs Dynamic Secrets

```
Static credentials (Jenkins credential store):
  Jenkins admin creates credential: aws-production-key
  Credential: AWS Access Key ID + Secret Access Key
  Stored encrypted in JENKINS_HOME/credentials.xml
  Never changes (until admin manually rotates)

  Risk:
    ❌ Long-lived: if leaked, attacker has indefinite access
    ❌ Many copies: credential in Jenkins + rotated every N months
    ❌ Audit trail: Jenkins logs which pipeline used it but not what API
calls
    ❌ Manual rotation: someone has to remember to rotate it
    ❌ Blast radius: one credential leak = all pipelines using it at risk

Dynamic secrets (Vault):
  Pipeline requests AWS credentials from Vault at build time
  Vault creates a new IAM user (or issues STS token) for this specific
request
  Credential has TTL: valid for 60 minutes, then Vault automatically
revokes it

  Benefits:
    ✅ Short-lived: leaked credential expires in minutes/hours
    ✅ Unique per build: each build gets different credentials
    ✅ Automatic rotation: Vault handles it — no manual process
    ✅ Audit trail: every request logged in Vault audit log (who, what,
when)
    ✅ Revocation: compromise detected → revoke all leases immediately
    ✅ Zero long-lived credentials in Jenkins
```

⚙️ Integration Method 1: HashiCorp Vault Plugin

```
Plugin ID: hashicorp-vault-plugin

Authentication methods (Jenkins → Vault):
  AppRole:    Role ID + Secret ID (service account approach) — most common
  Token:      Static Vault token (simple but defeats the purpose)
  Kubernetes: Jenkins pod SA token exchanged for Vault token — BEST for
K8s
  AWS IAM:    Jenkins IAM role → Vault AWS auth method
  JWT/OIDC:   JWT bearer token for OIDC flows
```

```
// ── METHOD 1: withVault step ──────────────────────────────────────
pipeline {
    agent any

    stages {
        stage('Deploy with Dynamic Credentials') {
            steps {
                withVault(
                    configuration: [
                        vaultUrl:         'https://vault.example.com',
                        vaultCredentialId: 'vault-approle-credentials',
// AppRole
                        engineVersion:     2   // KV v2 (or 1 for KV v1)
                    ],
                    vaultSecrets: [
                        [   // Read AWS credentials from Vault
                            path:        'secret/data/aws/production',
                            engineVersion: 2,
                            secretValues: [
                                [envVar: 'AWS_ACCESS_KEY_ID',
vaultKey: 'access_key'],
                                [envVar: 'AWS_SECRET_ACCESS_KEY',
vaultKey: 'secret_key'],
                                [envVar: 'AWS_REGION',
vaultKey: 'region']
                            ]
                        ],
                        [   // Read database credentials
                            path:        'database/creds/myapp-role',  //
Dynamic DB creds
                            engineVersion: 1,  // Dynamic secrets engine
uses KV v1 style
                            secretValues: [
                                [envVar: 'DB_USER',     vaultKey:
'username'],
                                [envVar: 'DB_PASSWORD', vaultKey:
'password']
                            ]
                        ]
                    ]
```

```
                ) {
                    // AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, DB_USER,
DB_PASSWORD
                    // are available as masked env vars in this block
                    sh """
                        aws eks update-kubeconfig --region \$AWS_REGION --
name production-cluster
                        helm upgrade --install myapp ./chart --namespace
production
                    """
                }
                // Credentials are automatically revoked when block exits
                // (if using dynamic secrets with Vault leasing)
            }
        }
    }
}
```

⚙️ Integration Method 2: Vault Kubernetes Auth (Best for K8s)

```
How Kubernetes auth works:
  1. Jenkins pod has a Kubernetes Service Account
  2. Jenkins sends the SA JWT token to Vault's /auth/kubernetes/login
endpoint
  3. Vault verifies the JWT against the K8s API (is this a valid SA
token?)
  4. Vault checks: is this SA authorized for role "jenkins-role"?
  5. Vault issues a Vault token (with TTL) to the Jenkins pod
  6. Jenkins uses Vault token to read secrets

  No static credentials! Jenkins authenticates with its K8s identity.
```

```
# ── VAULT SETUP: Configure Kubernetes auth ──────────────────────
# Enable Kubernetes auth method in Vault
vault auth enable kubernetes

# Configure it to validate against the K8s cluster Jenkins runs in
vault write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc.cluster.local:443" \

kubernetes_ca_cert=@/var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
    token_reviewer_jwt="$(cat
/var/run/secrets/kubernetes.io/serviceaccount/token)"

# Create a Vault policy for Jenkins builds
vault policy write jenkins-build - <<EOF
# Allow reading static secrets
path "secret/data/myapp/*" {
```

```
  capabilities = ["read"]
}
# Allow generating dynamic AWS credentials
path "aws/creds/myapp-deploy-role" {
  capabilities = ["read"]
}
# Allow generating dynamic database credentials
path "database/creds/myapp-role" {
  capabilities = ["read"]
}
EOF

# Create a Vault role that maps Jenkins SA to the policy
vault write auth/kubernetes/role/jenkins-build \
    bound_service_account_names="jenkins-build-sa" \
    bound_service_account_namespaces="jenkins-builds" \
    policies="jenkins-build" \
    ttl=1h                   # Vault token valid for 1 hour
```

```
// In pipeline: use Vault Agent Sidecar injection (no plugin needed)
// Kubernetes annotation on the build pod causes Vault Agent to inject
secrets as files

pipeline {
    agent {
        kubernetes {
            yaml '''
metadata:
  annotations:
    # Vault Agent Sidecar Injector annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "jenkins-build"
    vault.hashicorp.com/agent-inject-secret-aws: "aws/creds/myapp-deploy-
role"
    vault.hashicorp.com/agent-inject-template-aws: |
      {{- with secret "aws/creds/myapp-deploy-role" -}}
      export AWS_ACCESS_KEY_ID="{{ .Data.access_key }}"
      export AWS_SECRET_ACCESS_KEY="{{ .Data.secret_key }}"
      {{- end }}
    vault.hashicorp.com/agent-inject-secret-db: "database/creds/myapp-
role"
    vault.hashicorp.com/agent-inject-template-db: |
      {{- with secret "database/creds/myapp-role" -}}
      {{ .Data.username }}:{{ .Data.password }}
      {{- end }}
spec:
  serviceAccountName: jenkins-build-sa
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: deploy
```

```
    image: alpine/helm:3.14.0
    command: ["cat"]
    tty: true
'''
        }
    }

    stages {
        stage('Deploy') {
            steps {
                container('deploy') {
                    sh """
                        # Vault Agent wrote secrets to /vault/secrets/
                        # Source the AWS credentials file
                        . /vault/secrets/aws
                        # Now $AWS_ACCESS_KEY_ID and
$AWS_SECRET_ACCESS_KEY are set

                        # Read DB credentials
                        DB_CREDS=\$(cat /vault/secrets/db)
                        DB_USER=\$(echo "\$DB_CREDS" | cut -d: -f1)
                        DB_PASS=\$(echo "\$DB_CREDS" | cut -d: -f2)

                        helm upgrade --install myapp ./chart \
                            --namespace production \
                            --set aws.region=us-east-1 \
                            --set db.user="\$DB_USER"
                    """
                }
            }
        }
    }
}
```

⚙️ AWS Secrets Manager Integration

```
// AWS Secrets Manager: simpler for AWS-native environments
// No Vault to operate — secrets stored in AWS

stage('Get Secrets from AWS Secrets Manager') {
    steps {
        script {
            // Retrieve secret (Jenkins agent needs IAM role with
secretsmanager:GetSecretValue)
            def secret = sh(
                script: """
                    aws secretsmanager get-secret-value \
                        --secret-id myapp/production/database \
                        --region us-east-1 \
                        --query SecretString \
```

```
                        --output text
                """,
                returnStdout: true
            ).trim()

            // Parse JSON secret
            def secretData = readJSON text: secret
            env.DB_PASSWORD = secretData.password
            env.DB_USER     = secretData.username
            // These are now masked in logs (they're env vars with secret-
looking values)
        }
    }
}

// OR: Use AWS Secrets Manager Credentials Provider plugin
// Automatically syncs secrets from AWS Secrets Manager into Jenkins
credential store
// JCasC:
// unclassified:
//   awsCredentialsBinding:
//     region: us-east-1
//     listSecrets: true   # Auto-create Jenkins credentials from all
Secrets Manager secrets
```

⚙️ Secret Rotation Pattern

```
// Dynamic database credentials: a new username/password per build
// Vault creates a temporary PostgreSQL user, grants it role permissions,
// and revokes it when the lease expires

// In Vault:
// vault secrets enable database
// vault write database/config/myapp-db \
//     plugin_name=postgresql-database-plugin \
//     connection_url="postgresql://{{username}}:
{{password}}@db.example.com:5432/myapp" \
//     allowed_roles="myapp-role" \
//     username="vault-admin" \
//     password="vault-admin-pass"
//
// vault write database/roles/myapp-role \
//     db_name=myapp-db \
//     creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD
'{{password}}' VALID UNTIL '{{expiration}}'; GRANT app_role TO \"
{{name}}\";" \
//     default_ttl="1h" \
//     max_ttl="2h"

// In pipeline:
```

```
stage('Migration') {
    steps {
        withVault(
            configuration: [vaultUrl: 'https://vault.example.com',
                            vaultCredentialId: 'vault-approle'],
            vaultSecrets: [[
                path: 'database/creds/myapp-role',
                engineVersion: 1,
                secretValues: [
                    [envVar: 'FLYWAY_USER',     vaultKey: 'username'],
                    [envVar: 'FLYWAY_PASSWORD', vaultKey: 'password']
                ]
            ]]
        ) {
            // Each run: Vault creates a NEW PostgreSQL user (e.g., v-
myapp-random123)
            // with TTL = 1 hour — auto-deleted by Vault after the hour
            sh """
                flyway migrate \
                    -url=jdbc:postgresql://db.example.com:5432/myapp \
                    -user=\$FLYWAY_USER \
                    -password=\$FLYWAY_PASSWORD
            """
        }
        // User v-myapp-random123 is revoked by Vault after 1 hour
        // (or immediately if you call vault lease revoke)
    }
}
```

🔑 Key Concepts

Concept	Meaning
Dynamic secrets	Credentials generated on-demand with TTL — auto-revoked after use
AppRole auth	Vault auth: Role ID + Secret ID pair for service accounts
Kubernetes auth	Vault auth: K8s SA JWT token exchanged for Vault token — zero static creds
Secret lease	Time-bounded Vault secret allocation — automatically revoked at expiry
Vault Agent Sidecar	K8s injector that writes Vault secrets as files in pods
`withVault()`	Jenkins plugin step: authenticate to Vault, inject secrets as env vars
KV v2	Vault's key-value store with versioning — `secret/data/...` path prefix
Database secrets engine	Vault engine that creates temporary database users dynamically
AWS secrets engine	Vault engine that issues STS tokens or creates temporary IAM users

Concept	Meaning
Audit log	Vault logs every secret access — who requested what, when, outcome

---

💬 Short Crisp Interview Answer

> *"Jenkins+Vault integration replaces static credentials in Jenkins' credential store with dynamic secrets generated on-demand. The key benefit: short-lived credentials. A Vault-generated AWS STS token lasts 1 hour; a leaked Jenkins credential lasts until manually rotated (often months). For Kubernetes-based Jenkins, the best auth method is Kubernetes auth — the Jenkins pod's service account JWT is exchanged with Vault for a Vault token, with no static credentials anywhere. Secrets reach the pipeline two ways: the HashiCorp Vault plugin's `withVault()` step injects secrets as masked env vars; or the Vault Agent Sidecar Injector writes secrets as files in the pod via Kubernetes annotations. For database credentials specifically, Vault's database secrets engine creates a temporary PostgreSQL/MySQL user per build request — each pipeline run gets unique credentials that Vault automatically revokes when the lease expires. Full audit trail: every secret read is logged in Vault with the requester's identity."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Vault token renewal during long pipelines.** Vault tokens have a TTL. If a pipeline runs longer than the token's TTL (e.g., a 3-hour pipeline with a 1-hour Vault token), subsequent Vault reads fail with "permission denied — token expired." The Vault plugin handles renewal for the `withVault` block duration, but long-running stages outside the block may fail. Solution: use the Vault Agent Sidecar which handles renewal automatically.
- **AppRole Secret ID is itself a credential that needs protecting.** AppRole auth requires a Secret ID — which is essentially another static credential. Use "Response Wrapping" for Secret ID delivery (the Secret ID is wrapped in a single-use token) or switch to Kubernetes auth to eliminate the bootstrap credential problem entirely.
- **Vault Agent Sidecar file secrets aren't automatically masked in Jenkins logs.** Files at `/vault/secrets/aws` contain plaintext secrets. If you `cat /vault/secrets/aws` or `echo "$(cat /vault/secrets/aws)"` in a shell command, the secrets appear in logs. Read the files into variables and treat them as secrets: `DB_PASS=$(cat /vault/secrets/db-pass)` — but still don't `echo $DB_PASS`.
- **Dynamic database credentials require DB migration scripts to be idempotent.** If Vault creates user `v-build-42-abc` for a migration run, and the migration fails mid-way, the next retry gets user `v-build-42-xyz` (different credentials). Schema state must be fully idempotent or resume-safe, not dependent on the same user executing all steps.

Topic 6.7 — Jenkins + AWS/GCP/Azure

🔴 Advanced | Cloud-Native CI/CD

📌 What It Is — In Simple Terms

Cloud provider integrations connect Jenkins pipelines to cloud services: deploying containerized applications, pushing images to cloud registries (ECR/GCR/ACR), managing cloud infrastructure, and — critically — authenticating securely using cloud-native IAM rather than long-lived static credentials.

⚙️ AWS Integration

```
Core AWS services in Jenkins pipelines:
  ECR (Elastic Container Registry):   Private Docker image registry
  EKS (Elastic Kubernetes Service):   Managed Kubernetes
  ECS (Elastic Container Service):    Managed containers (non-K8s)
  S3:                                 Artifact storage
  Lambda:                             Serverless function deployment
  CodePipeline/CodeBuild:             AWS-native CI/CD (alternative to
Jenkins)

Authentication: IAM Roles (not access keys)
  Anti-pattern: static IAM access key + secret → Jenkins credential
    ❌ Key never expires, leaked key = persistent access

  Best practice: IAM Role attached to Jenkins EC2/EKS node
    Jenkins running on EC2: assign IAM role to the instance
    Jenkins running on EKS: use IRSA (IAM Roles for Service Accounts)
    → All AWS CLI/SDK calls on the Jenkins node automatically get
      temporary credentials from EC2/EKS metadata service
    → No credentials to store, rotate, or leak
```

```
// ── AWS ECR: Build and push Docker image ──────────────────────────
pipeline {
    agent { label 'aws-agent' }  // Agent on EC2 with IAM role, or EKS +
IRSA

    environment {
        AWS_ACCOUNT = '123456789012'
        AWS_REGION  = 'us-east-1'
        ECR_REPO    = "${AWS_ACCOUNT}.dkr.ecr.${AWS_REGION}.amazonaws.com"
        IMAGE_NAME  = "${ECR_REPO}/myapp"
        IMAGE_TAG   = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
            }
        }

        stage('Push to ECR') {
```

```
            steps {
                sh """
                    # Get temporary ECR login token (valid 12 hours)
                    # IAM role must have: ecr:GetAuthorizationToken
                    aws ecr get-login-password --region ${AWS_REGION} | \
                        docker login --username AWS \
                                     --password-stdin ${ECR_REPO}

                    # Ensure ECR repository exists
                    aws ecr describe-repositories \
                        --repository-names myapp \
                        --region ${AWS_REGION} 2>/dev/null || \
                    aws ecr create-repository \
                        --repository-name myapp \
                        --region ${AWS_REGION} \
                        --image-scanning-configuration scanOnPush=true

                    docker push ${IMAGE_NAME}:${IMAGE_TAG}

                    # Tag and push 'latest' for main branch
                    if [ "${env.BRANCH_NAME}" = "main" ]; then
                        docker tag ${IMAGE_NAME}:${IMAGE_TAG}
${IMAGE_NAME}:latest
                        docker push ${IMAGE_NAME}:latest
                    fi
                """
            }
        }

        stage('Deploy to EKS') {
            when { branch 'main' }
            steps {
                sh """
                    # Configure kubectl for EKS (using IAM role for auth)
                    aws eks update-kubeconfig \
                        --region ${AWS_REGION} \
                        --name production-cluster \
                        --alias production

                    helm upgrade --install myapp ./chart \
                        --namespace production \
                        --set image.repository=${IMAGE_NAME} \
                        --set image.tag=${IMAGE_TAG} \
                        -f chart/values-production.yaml \
                        --wait --atomic
                """
            }
        }

        stage('Deploy to ECS') {
            // Alternative: deploy to ECS Fargate
            when {
                expression { return env.DEPLOY_TARGET == 'ecs' }
            }
```

```
            steps {
                sh """
                    # Update ECS service to use new image
                    # First: update the task definition
                    TASK_DEF=\$(aws ecs describe-task-definition \
                        --task-definition myapp \
                        --query 'taskDefinition' \
                        --output json)

                    NEW_TASK_DEF=\$(echo \$TASK_DEF | python3 -c "
import sys, json
td = json.load(sys.stdin)
td['containerDefinitions'][0]['image'] = '${IMAGE_NAME}:${IMAGE_TAG}'
# Remove fields that can't be re-registered
for k in ['taskDefinitionArn','revision','status','requiresAttributes',
          'compatibilities','registeredAt','registeredBy']:
    td.pop(k, None)
print(json.dumps(td))
")
                    # Register new task definition revision
                    NEW_REVISION=\$(aws ecs register-task-definition \
                        --cli-input-json "\$NEW_TASK_DEF" \
                        --query 'taskDefinition.taskDefinitionArn' \
                        --output text)

                    # Update service to use new revision
                    aws ecs update-service \
                        --cluster production \
                        --service myapp \
                        --task-definition "\$NEW_REVISION" \
                        --force-new-deployment

                    # Wait for deployment to complete
                    aws ecs wait services-stable \
                        --cluster production \
                        --services myapp
                """
            }
        }

        stage('Deploy Lambda') {
            steps {
                sh """
                    # Package Lambda function
                    zip -r function.zip . -x "*.git*" -x
"node_modules/.cache/*"

                    # Deploy Lambda function (update code)
                    aws lambda update-function-code \
                        --function-name myapp-processor \
                        --zip-file fileb://function.zip \
                        --region ${AWS_REGION}

                    # Wait for update to complete
```

```
                        aws lambda wait function-updated \
                            --function-name myapp-processor \
                            --region ${AWS_REGION}

                        # Publish new version and update alias
                        VERSION=\$(aws lambda publish-version \
                            --function-name myapp-processor \
                            --description "Build ${env.BUILD_NUMBER}" \
                            --query 'Version' --output text)

                        aws lambda update-alias \
                            --function-name myapp-processor \
                            --name production \
                            --function-version "\$VERSION"
                    """
                }
            }
        }
    }
```

⚙️ GCP Integration

```
// GCP: Workload Identity (K8s SA → GCP SA — no key files)
// Jenkins on GKE with Workload Identity configured:
// K8s SA jenkins-build-sa → GCP SA
jenkins@myproject.iam.gserviceaccount.com

pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  serviceAccountName: jenkins-build-sa   # Has Workload Identity
annotation
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: gcloud
    image: google/cloud-sdk:latest
    command: ["cat"]
    tty: true
'''
        }
    }

    environment {
        GCP_PROJECT = 'my-project-id'
        GCR_HOST    = 'us-central1-docker.pkg.dev'
        REGION      = 'us-central1'
        GKE_CLUSTER = 'production-cluster'
```

```
        IMAGE_NAME  = "${GCR_HOST}/${GCP_PROJECT}/myapp/myapp"
        IMAGE_TAG   = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build & Push to Artifact Registry') {
            steps {
                container('gcloud') {
                    sh """
                        # Configure Docker to use Artifact Registry (no
key file needed with WI)
                        gcloud auth configure-docker ${GCR_HOST} --quiet

                        docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
                        docker push ${IMAGE_NAME}:${IMAGE_TAG}

                        # Sign image with Binary Authorization
                        gcloud beta container binauthz attestations sign-
and-create \
                            --project=${GCP_PROJECT} \
                            --artifact-
url="${IMAGE_NAME}:${IMAGE_TAG}@sha256:\$(docker inspect --format='{{index
.RepoDigests 0}}' ${IMAGE_NAME}:${IMAGE_TAG} | cut -d@ -f2)" \
                            --
attestor=projects/${GCP_PROJECT}/attestors/verified-builds \
                            --attestor-project=${GCP_PROJECT} \
                            --keyversion-project=${GCP_PROJECT} \
                            --keyversion-location=global \
                            --keyversion-keyring=attestor-keys \
                            --keyversion-key=verified-builds-key \
                            --keyversion=1
                    """
                }
            }
        }

        stage('Deploy to GKE') {
            steps {
                container('gcloud') {
                    sh """
                        # Get GKE credentials (uses Workload Identity - no
service account key file)
                        gcloud container clusters get-credentials
${GKE_CLUSTER} \
                            --region ${REGION} \
                            --project ${GCP_PROJECT}

                        helm upgrade --install myapp ./chart \
                            --namespace production \
                            --set image.repository=${IMAGE_NAME} \
                            --set image.tag=${IMAGE_TAG} \
                            --wait --atomic
                    """
                }
```

```
            }
        }
    }
}
```

⚙️ Azure Integration

```
// Azure: Managed Identity (no credentials to manage)
// Jenkins VM/AKS pod with Managed Identity assigned

pipeline {
    agent { label 'azure-agent' }

    environment {
        SUBSCRIPTION_ID = 'xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'
        RESOURCE_GROUP  = 'myapp-production'
        ACR_NAME        = 'mycompanyregistry'
        ACR_LOGIN       = "${ACR_NAME}.azurecr.io"
        AKS_CLUSTER     = 'production-aks'
        IMAGE_NAME      = "${ACR_LOGIN}/myapp"
        IMAGE_TAG       = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Login to ACR') {
            steps {
                sh """
                    # Login using Managed Identity (no credentials stored)
                    az login --identity
                    az acr login --name ${ACR_NAME}
                """
            }
        }

        stage('Build & Push to ACR') {
            steps {
                sh """
                    docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .
                    docker push ${IMAGE_NAME}:${IMAGE_TAG}
                """
            }
        }

        stage('Deploy to AKS') {
            when { branch 'main' }
            steps {
                sh """
                    # Get AKS credentials
                    az aks get-credentials \
                        --resource-group ${RESOURCE_GROUP} \
```

```
                        --name ${AKS_CLUSTER} \
                        --overwrite-existing

                    helm upgrade --install myapp ./chart \
                        --namespace production \
                        --set image.repository=${IMAGE_NAME} \
                        --set image.tag=${IMAGE_TAG} \
                        --wait --atomic
                '''
            }
        }
    }
}
```

🔑 Key Concepts — Cloud Auth Patterns

Cloud	Anti-pattern	Best Practice
AWS	IAM access key in Jenkins credential	IAM Role on EC2/EKS node (IRSA)
GCP	Service account JSON key file	Workload Identity (K8s SA → GCP SA)
Azure	Service Principal client secret	Managed Identity on VM/AKS node

```
All three clouds have the same pattern:
  ❌ Static key/secret → Long-lived → Must rotate manually → Blast radius
large
  ✅ Identity-based → Short-lived tokens from metadata service → No
rotation needed

  On EC2/GCE/Azure VM: metadata service at 169.254.169.254 provides temp
credentials
  On EKS/GKE/AKS: pod identity maps K8s SA to cloud IAM → same effect

  Result: aws/gcloud/az CLI "just works" with no credentials configured
```

💬 Short Crisp Interview Answer

> *"Cloud provider integrations cover image registry push, managed Kubernetes deployment, and serverless function updates. The most important design decision is authentication: never use static IAM access keys, GCP service account JSON files, or Azure service principal secrets stored in Jenkins. These are long-lived credentials that create rotation and leakage risk. The correct approach: use cloud-native identity binding. AWS: attach an IAM Role to the EC2 instance or use IRSA for EKS pods — the AWS CLI gets temporary credentials automatically from the metadata service. GCP: configure Workload Identity to map the K8s service account to a GCP service account — gcloud CLI authenticates automatically. Azure: use Managed Identity on the VM or AKS pod —* `az login --identity` *with no credentials needed. For container registry:* `aws ecr get-login-password |`

> *`docker login`, `gcloud auth configure-docker`, `az acr login` all get temporary tokens automatically when identity binding is configured. Zero static credentials anywhere."*

⚠ Tricky Edge Cases & Gotchas

- **ECR login token expires in 12 hours.** The `aws ecr get-login-password` token is valid for 12 hours. For long-running deployments (staged rollouts over multiple hours), re-authenticate before pushing additional images.
- **IRSA requires pod annotation AND service account annotation.** EKS IRSA needs: (1) the K8s ServiceAccount annotated with `eks.amazonaws.com/role-arn`, AND (2) the pod spec referencing that ServiceAccount, AND (3) the OIDC provider configured for the EKS cluster. Missing any one of the three means the pod falls back to the node's IAM role (often more permissive than desired).
- **`aws eks update-kubeconfig` modifies the local kubeconfig.** On shared agents running multiple concurrent builds, two parallel builds calling `update-kubeconfig` simultaneously can corrupt the kubeconfig. Use unique `--kubeconfig` paths per build: `aws eks update-kubeconfig --kubeconfig /tmp/kubeconfig-${BUILD_NUMBER}`.
- **GCP Workload Identity propagation delay.** After binding a K8s SA to a GCP SA, there's a 30-120 second propagation delay before the pod can authenticate. If you create the binding and immediately start a build, the first run may fail with permission denied. Plan for this in new cluster setup.

Topic 6.8 — Jenkins + Terraform

🔴 Advanced | Infrastructure as Code in Pipelines

📌 What It Is — In Simple Terms

Terraform manages cloud infrastructure (VMs, databases, networking, IAM, Kubernetes clusters) as code. Jenkins pipelines run Terraform to provision and update infrastructure — creating a CI/CD pipeline for infrastructure changes, not just application code. This introduces unique challenges: unlike application deployments, infrastructure changes can be destructive, have long execution times, require state management, and need careful change review before applying.

🔍 Why Terraform in Jenkins Pipelines

```
Without infrastructure pipeline:
  Developer: "I need a new RDS database for the auth service"
  Process: open ticket → ops team → click AWS console → create RDS
  Problems: slow, not version-controlled, no audit trail, hard to
reproduce

With infrastructure pipeline:
  Developer: create PR modifying terraform/auth-service/database.tf
  Jenkins: plan → shows exactly what will change → code review of the plan
  Approval: team lead approves the PR and the plan
```

```
  Jenkins: apply → database created with exact spec → state stored in S3
  Audit trail: Git commit + plan output + apply log = complete record
```

⚙️ Terraform State Management — The Critical Prerequisite

```
Terraform state (terraform.tfstate) tracks what infrastructure exists.
Without remote state: state on local disk → lost when agent pod is deleted

Remote state backends:
  AWS:   S3 bucket + DynamoDB table for state locking
  GCP:   Google Cloud Storage bucket
  Azure: Azure Blob Storage
  Terraform Cloud/Enterprise: built-in state management

S3 backend configuration:
```

```
# backend.tf — in the Terraform code
terraform {
  backend "s3" {
    bucket         = "mycompany-terraform-state"
    key            = "myapp/production/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true           # AES-256 encryption at rest
    dynamodb_table = "terraform-state-lock"   # Prevents concurrent
applies
  }

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"           # Pin provider version
    }
  }
}

# DynamoDB table for state locking (create this manually or via separate
Terraform):
# aws dynamodb create-table \
#     --table-name terraform-state-lock \
#     --attribute-definitions AttributeName=LockID,AttributeType=S \
#     --key-schema AttributeName=LockID,KeyType=HASH \
#     --billing-mode PAY_PER_REQUEST
```

⚙️ Complete Terraform Pipeline

```
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  serviceAccountName: jenkins-build-sa   # IRSA for AWS access
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: terraform
    image: hashicorp/terraform:1.7.0
    command: ["cat"]
    tty: true
    resources:
      requests: {cpu: "200m", memory: "256Mi"}
      limits:   {cpu: "500m", memory: "512Mi"}
  - name: infracost
    image: infracost/infracost:ci-0.10
    command: ["cat"]
    tty: true
'''
        }
    }

    options {
        timeout(time: 60, unit: 'MINUTES')
        // Never run concurrent Terraform applies against the same state
        lock(resource: 'terraform-production-state')
    }

    environment {
        TF_DIR          = 'terraform/production'
        TF_VAR_app_name = 'myapp'
        TF_VAR_region   = 'us-east-1'
        // TF_LOG = 'DEBUG'  // Enable for troubleshooting
    }

    stages {
        stage('Terraform Init') {
            steps {
                container('terraform') {
                    dir(env.TF_DIR) {
                        sh """
                            terraform init \
                                -backend-config="bucket=mycompany-
terraform-state" \
                                -backend-
config="key=myapp/production/terraform.tfstate" \
                                -backend-config="region=us-east-1" \
                                -reconfigure
                        """
                    }
                }
```

```
            }
        }

        stage('Terraform Validate & Format') {
            steps {
                container('terraform') {
                    dir(env.TF_DIR) {
                        sh """
                            terraform validate
                            terraform fmt -check -recursive
                        """
                    }
                }
            }
        }

        stage('Cost Estimation') {
            when { changeRequest() }   // Only on PRs
            steps {
                container('infracost') {
                    withCredentials([string(credentialsId: 'infracost-api-
key',
                                            variable:
'INFRACOST_API_KEY')]) {
                        dir(env.TF_DIR) {
                            sh """
                                infracost breakdown \
                                    --path . \
                                    --format json \
                                    --out-file /tmp/infracost.json

                                infracost diff \
                                    --path . \
                                    --format table
                            """
                        }
                    }
                }
            }
        }

        // ── TERRAFORM PLAN ──────────────────────────────────────────
        stage('Terraform Plan') {
            steps {
                container('terraform') {
                    dir(env.TF_DIR) {
                        sh """
                            terraform plan \
                                -out=tfplan \
                                -var-file=vars/production.tfvars \
                                -detailed-exitcode \
                                2>&1 | tee plan-output.txt

                            # Exit codes:
```

```
                            # 0 = no changes
                            # 1 = error
                            # 2 = there are changes (success with diff)
                        """
                    }
                }
                // Archive the plan for review and the apply step
                archiveArtifacts artifacts: "${TF_DIR}/plan-output.txt",
                                 allowEmptyArchive: false

                // Show the plan in build description
                script {
                    def planOutput = readFile("${TF_DIR}/plan-output.txt")
                    def summary = planOutput.find(/Plan: \d+ to add, \d+
to change, \d+ to destroy\./)
                    currentBuild.description = summary ?: "Plan complete"
                }
            }
        }

        stage('Plan Review Gate') {
            when {
                allOf {
                    branch 'main'
                    // Only gate if there are actual changes
                    expression {
                        return !readFile("${TF_DIR}/plan-
output.txt").contains('No changes.')
                    }
                }
            }
            steps {
                // Show the plan output in the approval dialog
                script {
                    def planOutput = readFile("${TF_DIR}/plan-output.txt")
                    // Truncate to last 4000 chars to fit in input dialog
                    def planSummary = planOutput.length() > 4000 ?
                        "...(truncated)\n" +
planOutput.substring(planOutput.length() - 4000) :
                        planOutput

                    timeout(time: 24, unit: 'HOURS') {
                        input(
                            message: "Review Terraform
Plan:\n\n${planSummary}\n\nApply changes to PRODUCTION?",
                            ok:      'Apply Changes',
                            submitter: 'infra-team,sre-leads'
                        )
                    }
                }
            }
        }

        // ── TERRAFORM APPLY ──────────────────────────────────────────
```

```
        stage('Terraform Apply') {
            when { branch 'main' }
            steps {
                container('terraform') {
                    dir(env.TF_DIR) {
                        sh """
                            # Apply the saved plan (guarantees what was
reviewed is what's applied)
                            terraform apply \
                                -auto-approve \
                                tfplan \
                                2>&1 | tee apply-output.txt
                        """
                    }
                }
                archiveArtifacts artifacts: "${TF_DIR}/apply-output.txt",
                                 allowEmptyArchive: false
            }
            post {
                failure {
                    container('terraform') {
                        dir(env.TF_DIR) {
                            // Check if state is locked after failure
                            sh 'terraform state list 2>&1 | head -20 ||
true'
                        }
                    }
                }
            }
        }

        stage('Terraform Output') {
            when { branch 'main' }
            steps {
                container('terraform') {
                    dir(env.TF_DIR) {
                        sh """
                            terraform output -json > /tmp/tf-outputs.json
                            cat /tmp/tf-outputs.json
                        """
                        // Use outputs in subsequent steps:
                        script {
                            def outputs = readJSON file: '/tmp/tf-
outputs.json'
                            env.RDS_ENDPOINT   =
outputs.rds_endpoint?.value
                            env.EKS_CLUSTER_ID =
outputs.eks_cluster_id?.value
                        }
                    }
                }
            }
        }
    }
```

```
    post {
        always {
            // Unlock: the lock() step in options unlocks automatically
            // But if Terraform left a state lock, manually unlock:
            script {
                try {
                    container('terraform') {
                        dir(env.TF_DIR) {
                            sh '''
                                # Check for stale locks
                                LOCK_ID=$(terraform force-unlock -list
2>/dev/null | grep LockID | awk '{print $2}' || echo "")
                                if [ -n "$LOCK_ID" ]; then
                                    echo "Found stale lock: $LOCK_ID -
releasing"
                                    # terraform force-unlock "$LOCK_ID"
(requires manual confirmation in prod)
                                fi
                            '''
                        }
                    }
                } catch (Exception e) {
                    echo "Lock check failed (non-fatal): ${e.message}"
                }
            }
        }
    }
}
```

⚙️ Terraform Drift Detection — Scheduled Pipeline

```
// Scheduled pipeline: detect configuration drift
// (someone manually changed AWS console → Terraform state no longer
matches reality)

pipeline {
    agent { kubernetes { yaml '...' } }

    triggers {
        cron('H 8 * * 1-5')   // Run every weekday morning
    }

    stages {
        stage('Drift Detection') {
            steps {
                container('terraform') {
                    dir('terraform/production') {
                        sh 'terraform init -reconfigure'
                        script {
```

```
                            def exitCode = sh(
                                script: 'terraform plan -detailed-exitcode
-refresh-only 2>&1 | tee drift-report.txt',
                                returnStatus: true
                            )
                            if (exitCode == 2) {
                                // Exit code 2 = differences detected
                                def driftReport = readFile('drift-
report.txt')
                                currentBuild.result = 'UNSTABLE'
                                slackSend(
                                    channel: '#infra-alerts',
                                    color:   'warning',
                                    message: "⚠ Terraform drift detected
in production!\n" +
                                             "Someone may have made manual
changes.\n" +
                                             "Review: ${env.BUILD_URL}"
                                )
                            } else if (exitCode == 0) {
                                echo "No drift detected ✅"
                            } else {
                                error("terraform plan failed with exit
code ${exitCode}")
                            }
                        }
                    }
                }
            }
        }
    }
}
```

⚙️ Preventing Destructive Operations

```
// Safeguard: check the plan for destructive operations before applying

stage('Check for Destructive Changes') {
    steps {
        container('terraform') {
            dir(env.TF_DIR) {
                script {
                    def planOutput = readFile('plan-output.txt')

                    // Parse the summary line
                    def destroyMatch = planOutput =~ /(\d+) to destroy/
                    def destroyCount = destroyMatch ? destroyMatch[0]
[1].toInteger() : 0

                    echo "Resources to destroy: ${destroyCount}"
```

```
                    if (destroyCount > 0) {
                        // Flag for extra approval
                        env.HAS_DESTRUCTIVE_CHANGES = 'true'
                        currentBuild.description = "⚠ ${destroyCount}
resources to DESTROY — ${currentBuild.description}"

                        // Require additional approver for destructive
changes
                        if (destroyCount > 5) {
                            error("Refusing to automatically gate: plan
destroys ${destroyCount} resources. Manual review required.")
                        }
                    }
                }
            }
        }
    }
}

stage('Destructive Change Approval') {
    when {
        expression { return env.HAS_DESTRUCTIVE_CHANGES == 'true' }
    }
    steps {
        timeout(time: 4, unit: 'HOURS') {
            input(
                message:   "⚠ WARNING: This plan DESTROYS
resources!\nCheck the archived plan-output.txt. Are you sure?",
                ok:        'Yes, I understand — Apply Anyway',
                submitter: 'infra-leads'   // Only infra leads can approve
destructive ops
            )
        }
    }
}
```

🔑 Key Concepts

Concept	Meaning
Remote state	Terraform state stored in S3/GCS/Azure Blob — persists across pipeline runs
State locking	DynamoDB/GCS lock prevents concurrent `terraform apply` on same state
`terraform plan -out=tfplan`	Save plan to file — guarantees apply uses exactly what was reviewed

Concept	Meaning
`-detailed-exitcode`	Exit 0=no changes, 1=error, 2=changes — machine-parseable plan result
Lockable Resources plugin	Jenkins lock() step prevents concurrent pipeline runs on same TF state
Drift detection	`terraform plan -refresh-only` — detect manual changes outside Terraform
`-auto-approve`	Skip interactive confirmation — required in automated pipelines
Workspace	Terraform workspace = separate state per environment (dev/staging/prod)
Infracost	Tool that estimates cloud cost impact of Terraform changes
`force-unlock`	Release a stuck Terraform state lock (use carefully)

💬 Short Crisp Interview Answer

> *"Jenkins+Terraform creates a CI/CD pipeline for infrastructure changes. The pipeline: init → validate/fmt → plan → human review → apply. The most critical design decisions: state must be remote (S3+DynamoDB for AWS — never local disk that dies with the agent pod), and concurrent applies must be prevented (Jenkins Lockable Resources plugin + Terraform's DynamoDB state locking are independent locks that together prevent race conditions). The `terraform plan -out=tfplan` saves the plan to a file — the apply step uses `terraform apply tfplan` which guarantees the exact plan that was reviewed is what gets applied, not a re-planned potentially different operation. For destructive changes, I add a separate parsing stage that reads the plan output, counts `N to destroy`, and routes to a separate high-privilege approval gate if any resources are being destroyed. Drift detection runs on a schedule: `terraform plan -detailed-exitcode -refresh-only` exits with code 2 if infrastructure drifted from the Terraform state — alerting the ops team that someone made manual changes."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`terraform apply` without `-out=tfplan` is dangerous in pipelines.** If you run `plan` then later `apply` without the plan file, Terraform re-plans at apply time. Between plan and apply, someone could merge another PR that changes the Terraform code — the apply then executes a DIFFERENT plan than what was reviewed. Always save the plan: `plan -out=tfplan` → review → `apply tfplan`.
- **State lock must be explicitly managed.** If a Jenkins build is killed (OOMKilled, timeout, agent crash) mid-apply, the DynamoDB state lock may remain. Terraform will refuse subsequent operations with "Error acquiring the state lock." Manual intervention: `terraform force-unlock <LOCK_ID>`. Add a lock check to your pipeline's `post { always {} }` block.
- **Terraform workspace ≠ environment isolation for state.** `terraform workspace new production` creates a separate state key in the same S3 bucket. It's NOT the same as having a separate backend config per environment. For strict environment isolation, use different S3 keys/buckets per environment in the backend config.

- **Provider credentials in the Terraform container.** The Terraform container needs AWS/GCP/Azure credentials. On Kubernetes with IRSA/Workload Identity, the pod's service account provides these automatically. But the Terraform container must be aware of this — the `terraform` image doesn't have the cloud CLI installed, so environment variables (`AWS_ROLE_ARN`, `AWS_WEB_IDENTITY_TOKEN_FILE`) are how credentials flow in.
- **Long apply times block the pipeline executor.** Large Terraform applies (provisioning an RDS Multi-AZ instance = 15-20 minutes) hold the build pod running. This is normal — Kubernetes pod agents handle long builds fine. Set `activeDeadlineSeconds` high enough in the pod spec.

📊 Category 6 Summary — Quick Reference

Topic	Core Concept	Interview Priority
6.1 Jenkins + Git	Webhooks > polling; SSH keys/GitHub App auth; shallow clone; `BRANCH_NAME` vs `GIT_BRANCH`	⭐⭐⭐
6.2 Jenkins + Docker ⚠	Multi-stage Dockerfile; Kaniko > DinD; layer caching; tag strategy	⭐⭐⭐⭐
6.3 Jenkins + Kubernetes	Helm `--atomic`; minimal RBAC service account; canary via Argo Rollouts; GitOps	⭐⭐⭐⭐⭐
6.4 Jenkins + SonarQube	Async analysis + `waitForQualityGate`; coverage first; webhook required	⭐⭐⭐⭐
6.5 Jenkins + Nexus/Artifactory	Proxy + hosted repos; generate `settings.xml` dynamically; artifact traceability	⭐⭐⭐
6.6 Jenkins + Vault ⚠	Dynamic secrets; K8s auth > AppRole; Vault Agent Sidecar; dynamic DB creds	⭐⭐⭐⭐⭐
6.7 Jenkins + Cloud	IAM Role/Workload Identity/Managed Identity over static keys; ECR/GCR/ACR login	⭐⭐⭐⭐⭐
6.8 Jenkins + Terraform	Remote state; save plan + apply plan; lock; drift detection; destructive change gate	⭐⭐⭐⭐⭐

🔑 The Mental Model for Category 6

```
Every integration has TWO questions:
  1. How does Jenkins AUTHENTICATE to this system?
  2. What does the PIPELINE STEP look like?

AUTHENTICATION HIERARCHY (most secure → least secure):
  ✅ Cloud-native identity (IAM Role, Workload Identity, Managed Identity)
  ✅ Dynamic secrets (Vault — short-lived, auto-rotated)
  ✅ Short-lived tokens (ECR login, GitHub App tokens)
```

```
  ⚠ Long-lived API tokens in Jenkins credentials (rotate regularly)
  ❌ Hardcoded credentials in Jenkinsfile / settings.xml

DOCKER: The two questions interviewers test
  "Are you building WITH Docker (kaniko/docker CLI) or running ON Docker
(K8s plugin)?"
  "DinD vs Kaniko?" → Always Kaniko in Kubernetes (no privileged required)

SONARQUBE: The async trap
  Analysis is async → MUST waitForQualityGate() in separate stage
  Coverage file must exist BEFORE sonar:sonar runs
  SonarQube webhook required for waitForQualityGate to receive callback

VAULT: The two mechanisms
  Plugin: withVault() → secrets as env vars (simpler, good for most cases)
  Sidecar Injector: files at /vault/secrets/ (better for K8s, handles
renewal)
  K8s auth: zero static credentials → preferred over AppRole

TERRAFORM: Three non-negotiables
  1. Remote state (never local disk)
  2. Save plan → review → apply the saved plan (not re-plan)
  3. Locking (Jenkins lock() + DynamoDB state lock = two independent
safety locks)
```

🧪 Self-Quiz — 10 Interview Questions

1. Why is `pollSCM` considered an anti-pattern? What's the correct alternative and why?
2. You need to build a Docker image inside a Kubernetes pod-based Jenkins build. DinD, DooD, or Kaniko — which do you use and why?
3. Walk through the `waitForQualityGate()` mechanism. What must exist for it to work, and what happens if you call it in the same stage as the analysis?
4. Your Terraform pipeline: Plan runs, infra team reviews, approves, then Apply runs. What's wrong with this pipeline and how do you fix it?
5. What is Vault Kubernetes auth? How does it eliminate the "bootstrap credential problem" that AppRole has?
6. A Jenkins pipeline running on EKS needs to push to ECR. You've been told "use an IAM user with an access key." Why is this wrong? What's the correct architecture?
7. `helm upgrade --atomic` fails mid-deploy. What happens? What state is the cluster in? What does the Helm history look like?
8. A SonarQube quality gate check always times out after 5 minutes. List three possible causes.
9. Why must concurrent Terraform applies be prevented? How do you prevent them in Jenkins?

10. What's the difference between Nexus proxy repositories and hosted repositories? Give a concrete example of each in a Java microservice build pipeline.

Next: Category 7 — Security in CI/CD Or: "quiz me" to test yourself on Category 6

> **Document:** Category 6 Complete | Jenkins Integrations **Coverage:** 8 topics | Beginner → Advanced **Key insight:** Every integration is a tradeoff between security, speed, and operational complexity. The authentication design is always the most important decision.